

Merge and Code Review

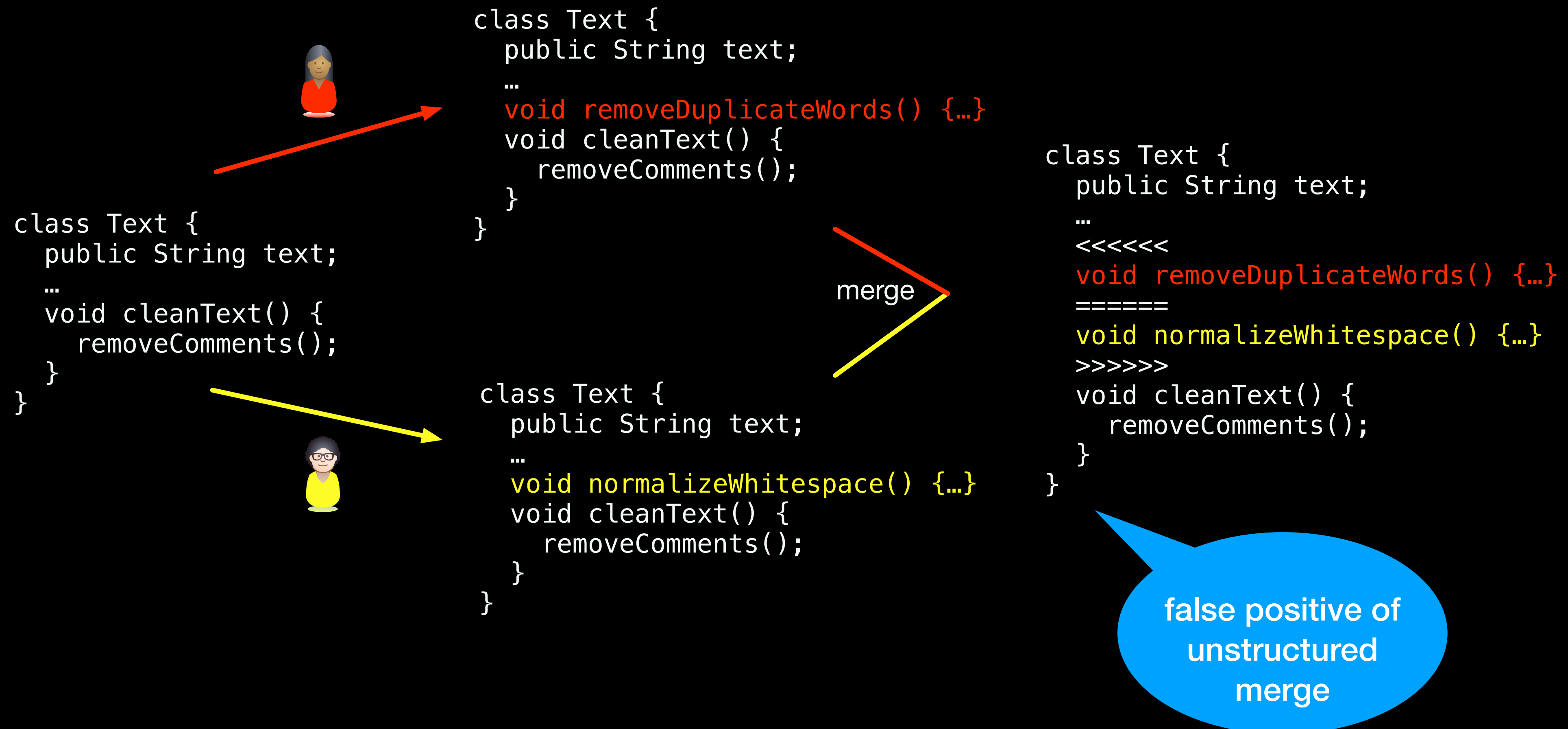
Paulo Borba
Informatics Center
Federal University of Pernambuco

pauloborba.cin.ufpe.br

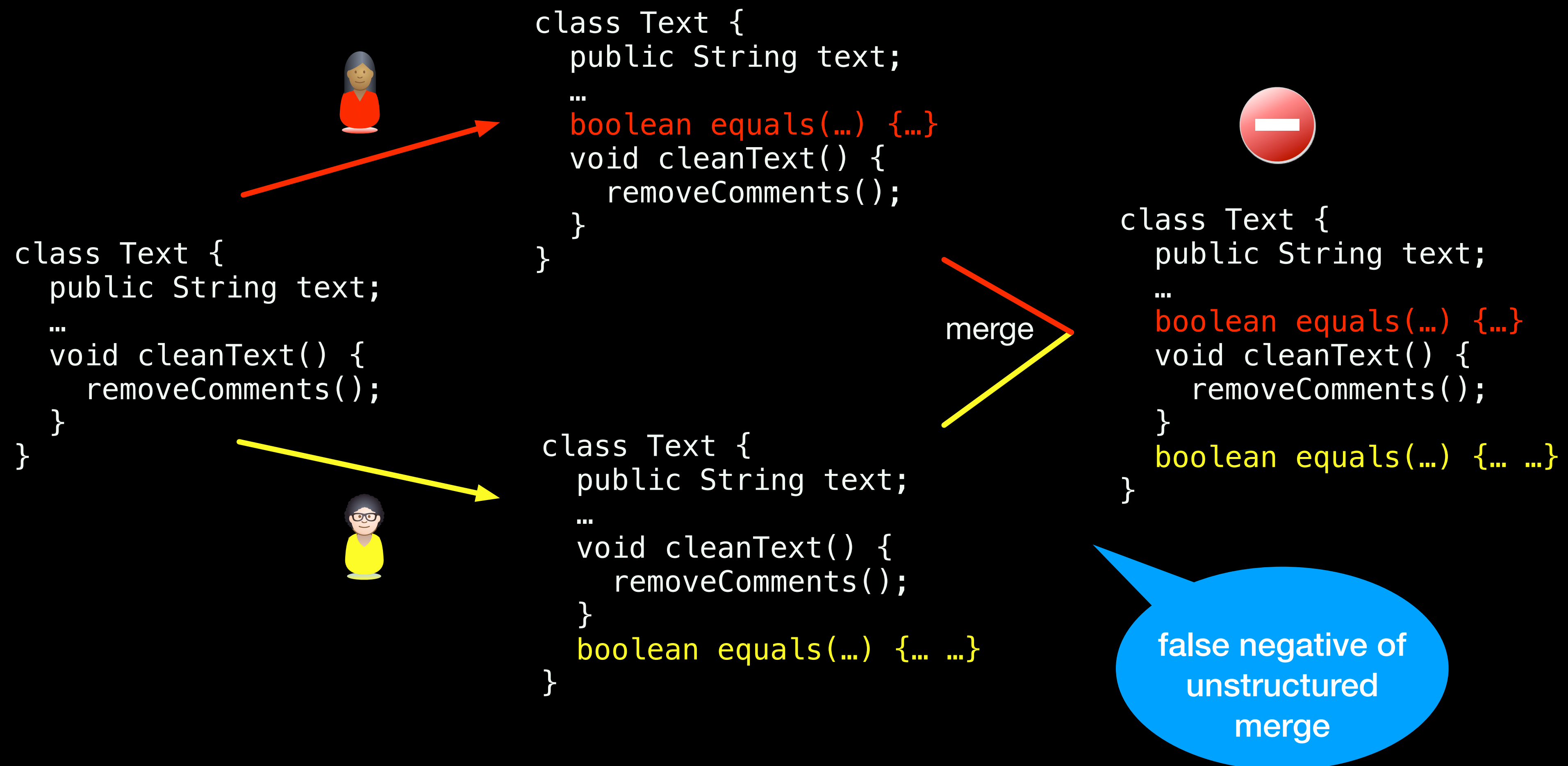
Structured merge

Trying to avoid unstructured merge
false positives and negatives

Developers waste time by manually resolving conflicts that could be automatically solved

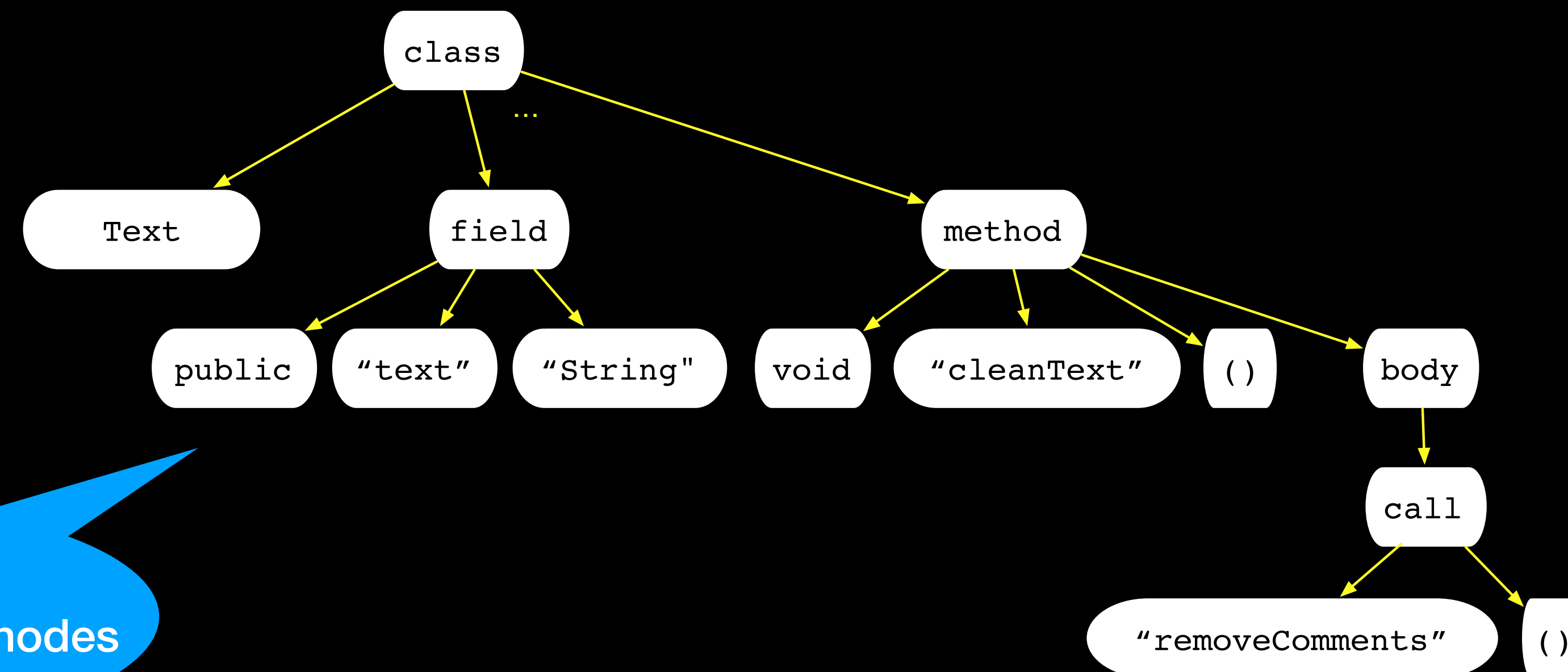


Current merge tools might integrate conflicting changes without warning developers

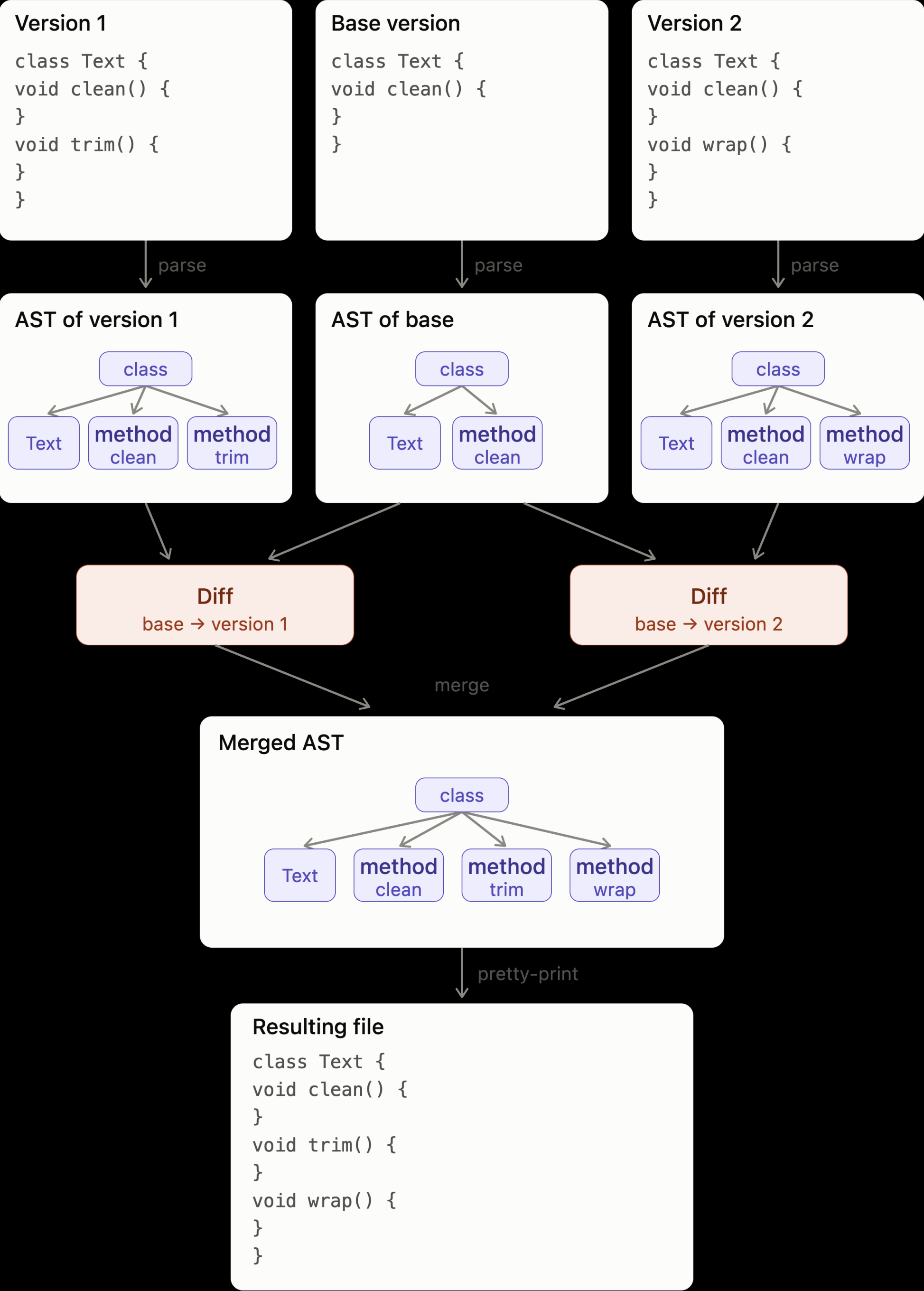


Structured merge works with ASTs, not a sequence of lines

```
class Text {      \n
  public String text; \n
  ...           \n
  void cleanText() { \n
    removeComments(); \n
  }             \n
}
```



no conflicts for
changes in different nodes



File versions =>

ASTs of each version =>

merged AST =>

Merged file

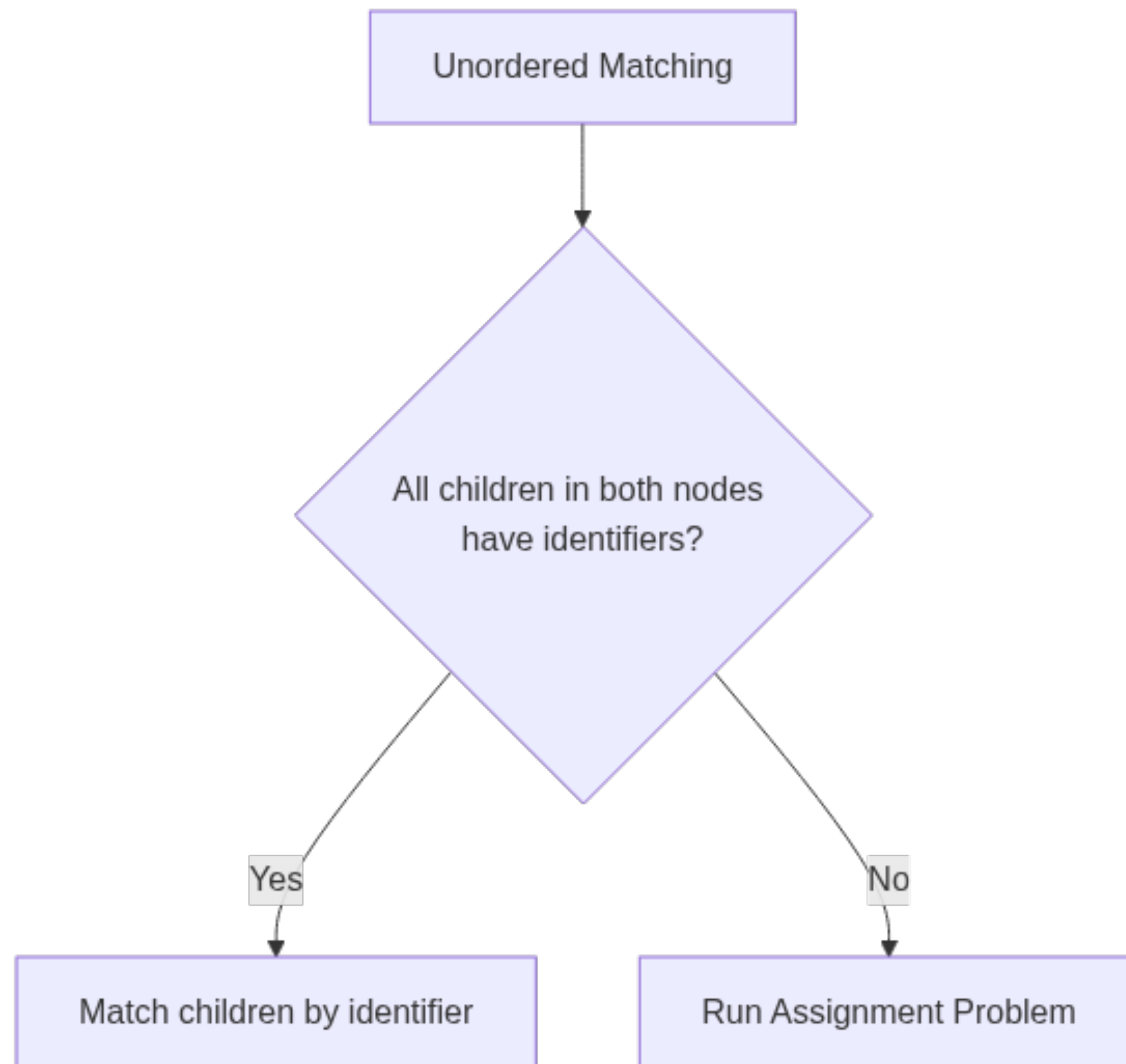
File versions =>

CSTs of each version =>

merged CST =>

Merged file

Unordered Matching: previously, algorithms were exclusive



Left

```
void speak() {}
```

```
void say_hello() {}
```

```
void cheer() {}
```

```
static {
}
```

Right

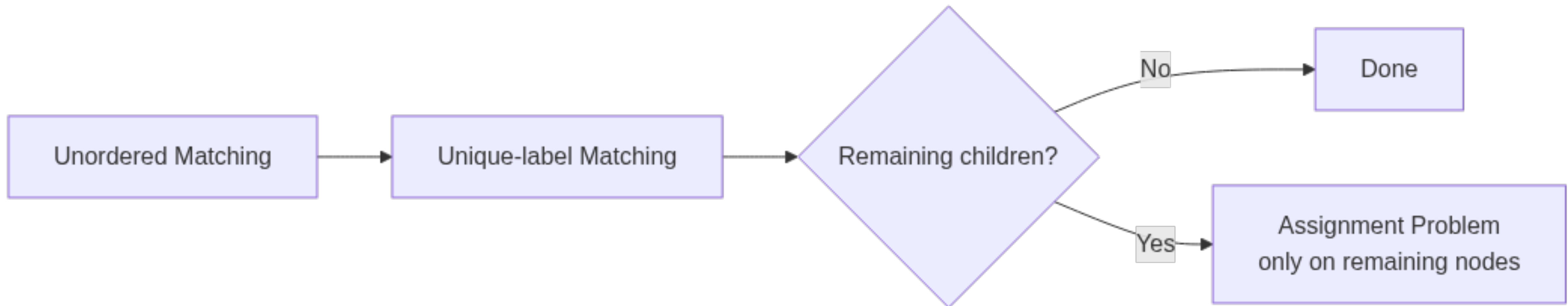
```
void speak() {}
```

```
void say_hello() {}
```

```
void boo() {}
```

Had to use assignment problem even if only one node is unlabelled

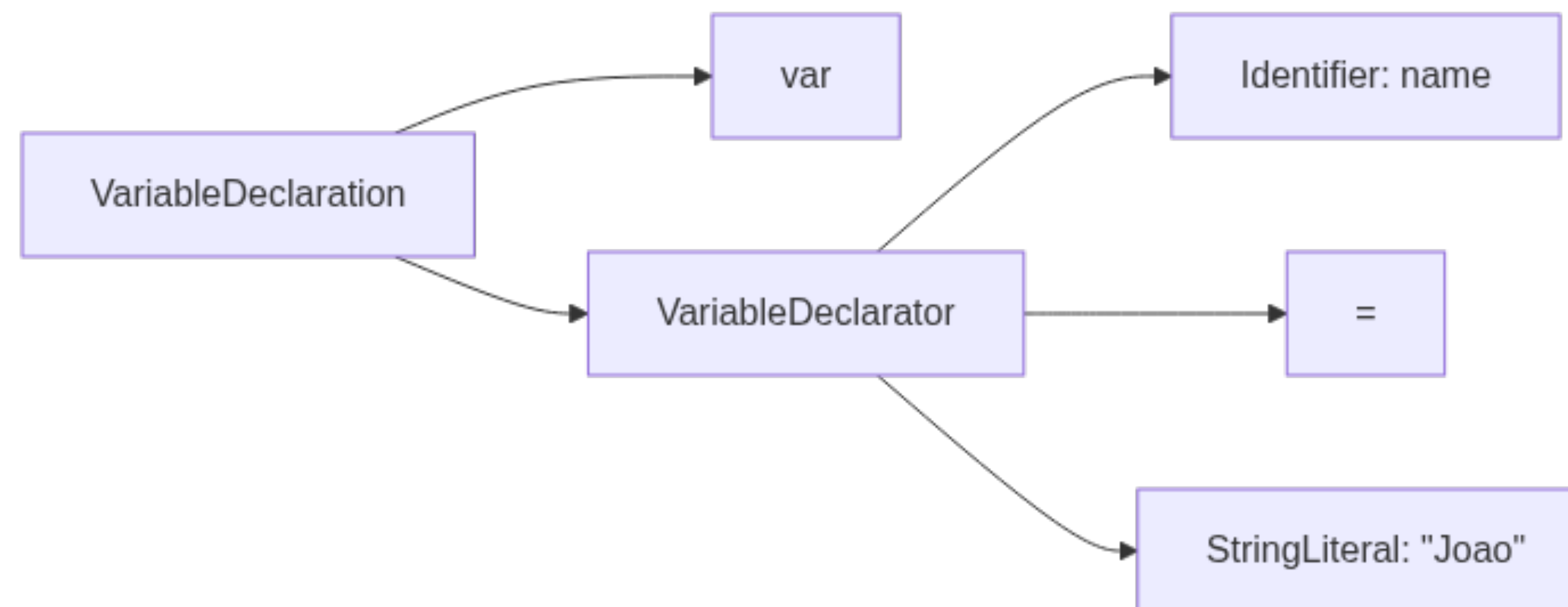
Unordered Matching: combining strategies to reduce search space in assignment problem



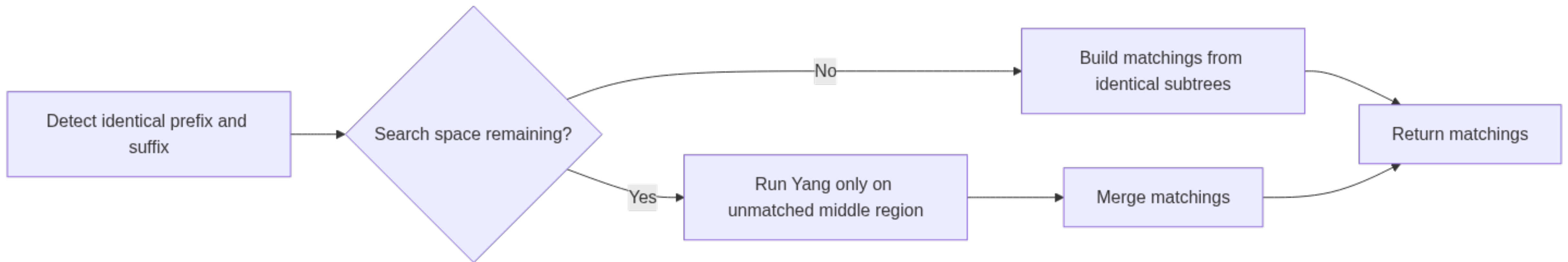
On average: significant reduction on the assignment problem search space, thus yielding to faster execution

Ordered Matching: previously, only Yang Algorithm

- A quadratic dynamic programming algorithm to compute matchings
- First optimization: add another algorithm that walk the trees together and stops if they are identical (linear time).



Combining algorithms: use tree walk to pre-compute prefix and suffix and reduce Yang search space



Left: A B C **X Y** D E F

Prefix: A B C

Yang comparing
X Y and Z W

Right: A B C **Z W** D E
F

Suffix: D E F

Source code edits are usually localized,
making this optimization effective in practice

Left: A B C **X Y** D E F

Prefix: A B C

Yang comparing
X Y and Z W

Right: A B C **Z W** D E
F

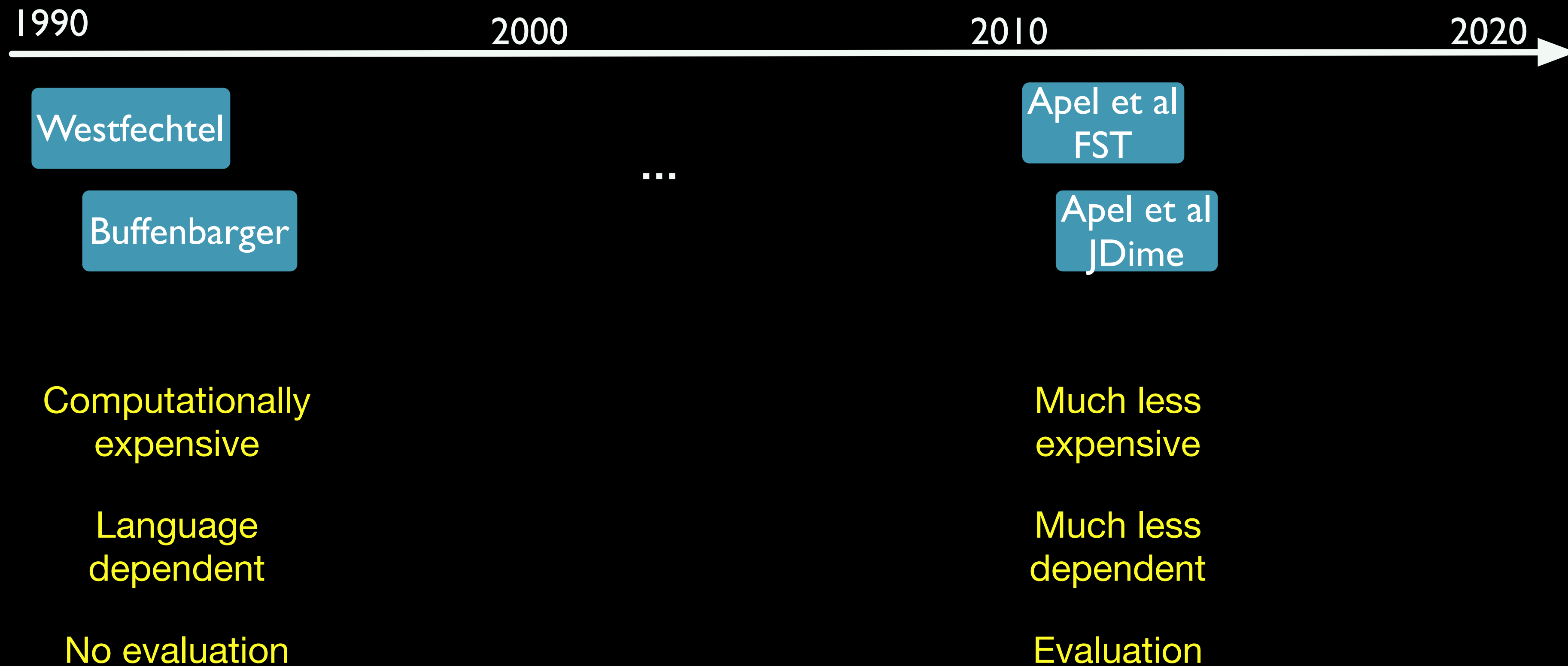
Suffix: D E F

Instead of running Yang over an 8x8 matrix, we only need to run over a 2x2. Given the quadratic complexity of Yang it significantly reduces time.

Other minor optimizations reduce constant factors without changing algorithmic behavior

Optimization	Motivation
Memoize subtree size computation	Avoid repeated traversals
Use rustc-hash hasher create	Faster HashMap operations
Avoid repeated lists resizes	Reduce allocations
Capacity reservation in lists	
Early return on unitary matches	Avoid unnecessary Yang execution
Stop when one side is fully consumed	Reduce recursion

Structured merge tools timeline



B. Westfechtel. Structure-Oriented Merging of Revisions of Software Documents. SCM 1991.

J. Buffenbarger. Syntactic Software Merging. SCM 1995.

S. Apel, J. Liebig, B. Brandl, C. Lengauer, and C. Kästner. Semistructured Merge: Rethinking Merge in Revision Control Systems. ESEC/FSE 2011.

S. Apel, O. Leßenich, and C. Lengauer. Structured Merge with Auto-Tuning: Balancing Precision and Performance. ASE 2012.

The Impact of Language Independence on Structured Merge Accuracy and Efficiency

João Pedro Henrique Santos Duarte
Paulo Borba
Guilherme Cavalcanti

LASTMERGE – A language-agnostic structured tool for code integration

João Pedro Duarte, Paulo Borba, Guilherme Cavalcanti

Abstract—Unstructured line-based merge tools are widely used in practice. Structured AST-based merge tools show significantly improved merge accuracy, but are rarely used in practice because they are language specific and costly, consequently not being available for many programming languages. To improve merge accuracy for a wide range of languages, we propose LASTMERGE, a *generic* structured merge tool that can be configured through a thin interface that significantly reduces the effort of supporting structured merge. To understand the impact that *generic* structured merge might have on merge accuracy and performance, we run an experiment with four structured merge tools: two Java specific tools, *jDine* and *Spork*, and their *generic* counterparts, respectively LASTMERGE and *Mergiraf*. Using each tool, we replay merge scenarios from a significant dataset, and collect data on runtime, behavioral divergences, and merge accuracy. Our results show no evidence that *generic* structured merge significantly impacts merge accuracy. Although we observe a difference rate of approximately 10% between the Java specific tools and their *generic* counterparts, most of the differences stem from implementation details and could be avoided. We find that LASTMERGE reports 15% fewer false positives than *jDine* while *Mergiraf* misses 42% fewer false negatives than *Spork*. Both *generic* tools exhibit comparable runtime performance to the state of the art language specific implementations. These results suggest that *generic* structured merge tools can effectively replace language-specific ones, paving the way for broader adoption of structured merge in industry.

[12]. Despite these advances, structured tools remain largely absent from current practice especially for two reasons. First, as they strongly rely on the syntax and semantics of specific programming languages, structured tools proposed so far are language specific; substantial implementation and maintenance effort is needed for supporting a new language. Second, for being costly, not many languages are supported by structured tools, which hinders adoption by developers who routinely work with multiple languages, as often needed in practice.

To reduce these barriers and improve merge accuracy for a wide range of programming languages, we propose LASTMERGE, a *generic* structured merge tool that can be easily configurable for each language. It relies on a core merge engine that operates over generic trees produced by an extensible and fast parser framework [13] that has been instantiated for more than 350 languages, and is in production at GitHub. By feeding the engine with a high level description of language specific aspects (such as node labelling and restrictions to permutation of node children) that are known to be relevant for structured merge, developers can easily adapt LASTMERGE for new languages, or refine support for existing ones. This thin configuration interface significantly reduces the effort of having structured merge for a wide range of languages.

João e Paulo introduzem diferentes modificações em um mesmo método de uma classe

Original

```
public class Account {  
    void debit(int amount) {  
        // ...  
    }  
}
```

João

```
public class Account {  
    public void debit(int amount) {  
        // ...  
    }  
}
```

Paulo

```
public class Account {  
    static void debit(int amount) {  
        // ...  
    }  
}
```

Mas, quando integramos essas modificações...

Merge

```
public class Account {  
<<<<<<<<  
    public void debit(int amount) {  
=====  
        static void debit(int amount) {  
>>>>>>>>  
            // ...  
        }  
    }  
}
```


Acabamos esbarrando em um conflito de merge

Merge

```
public class Account {
```

```
<<<<<<<<
```

```
    public void debit(int amount) {
```

```
=====
```

```
    static void debit(int amount) {
```

```
>>>>>>>>
```

```
    // ...
```

```
}
```

```
}
```

Em Java, modificadores podem ter a ordem permutada

Account.java

```
public class Account {  
    <<<<<<<<  
        public void debit(int amount) {  
        =====  
        static void debit(int amount) {  
        >>>>>>>>  
        // ...  
    }  
}
```



Account.java

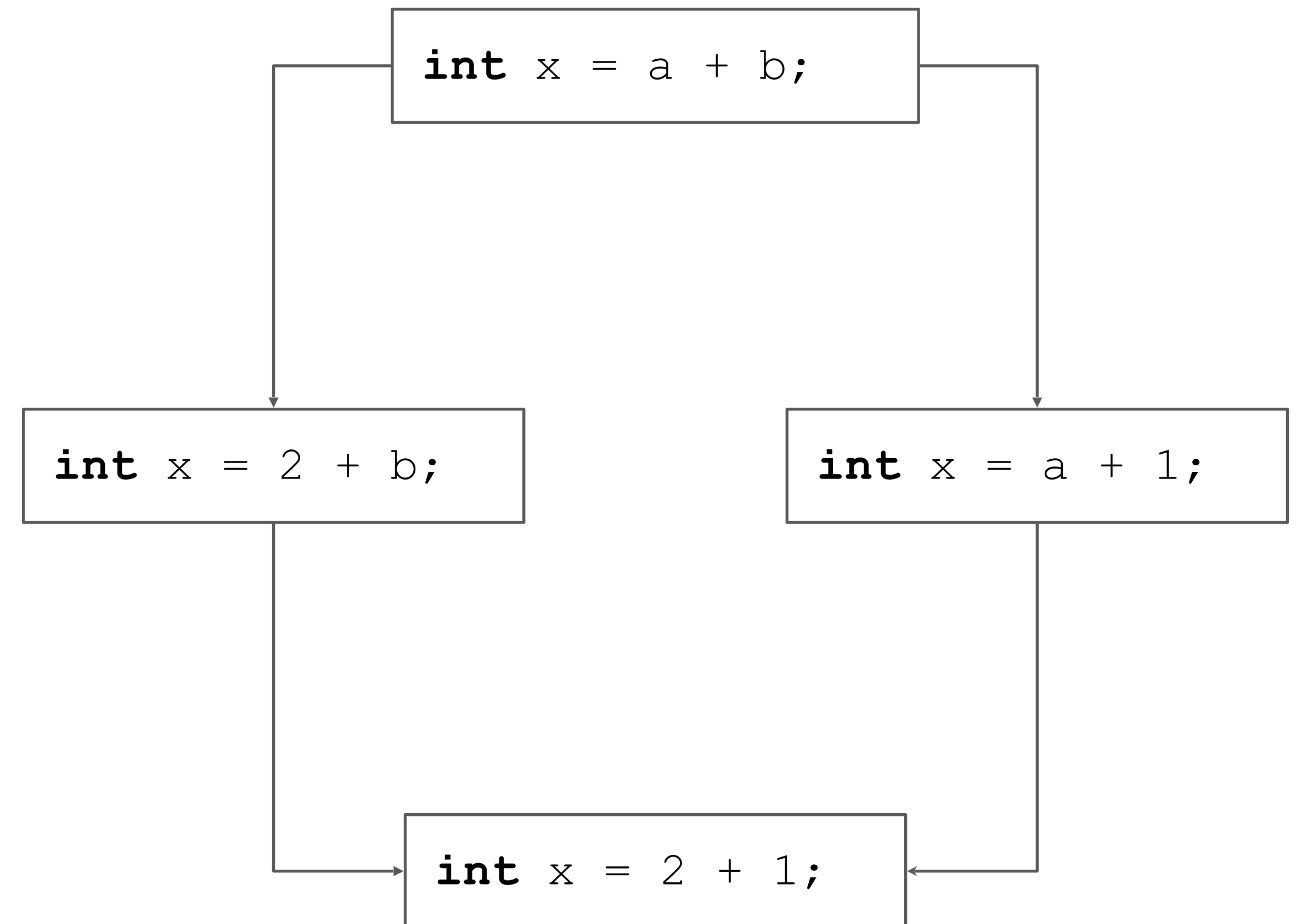
```
public class Account {  
    public static void debit(int  
amount) {  
        // ...  
    }  
}
```

Falso
positivo

Por que estes falsos positivos acontecem?

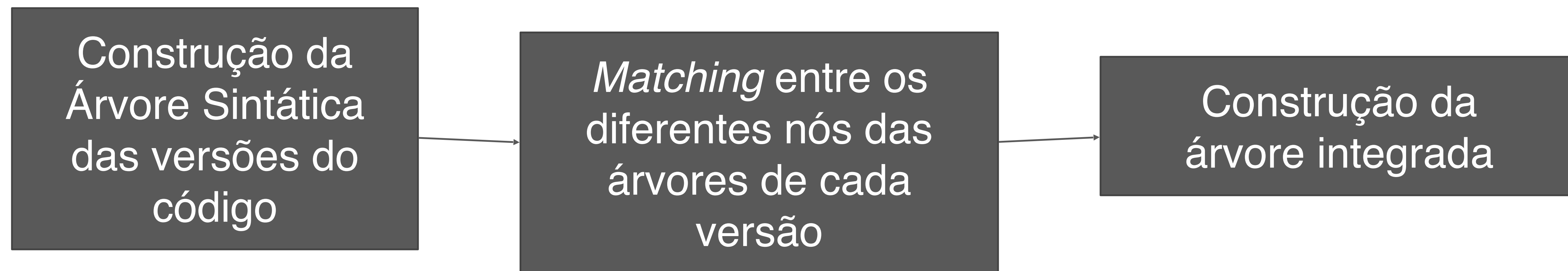
Estratégias convencionais consideram apenas a diferença de conteúdo entre as linhas durante o processo de merge.

**Mas, código não é uma mera sequência de caracteres e linhas.
Tem estrutura bem definida!**



Considerando a estrutura do programa

Enxergar os arquivos sendo integrados de maneira mais abstrata.



Ferramentas estruturadas já existem e tem resultados positivos (jDime).

- Redução no número de conflitos reportados em quase 30% [Seibt et al, 2021]

Por conhecer a estrutura da linguagem, merge estruturado resolve conflitos que ferramentas tradicionais não conseguem

João

```
public class Account {  
    public void debit(int amount) {  
        // ...  
    }  
}
```

Paulo

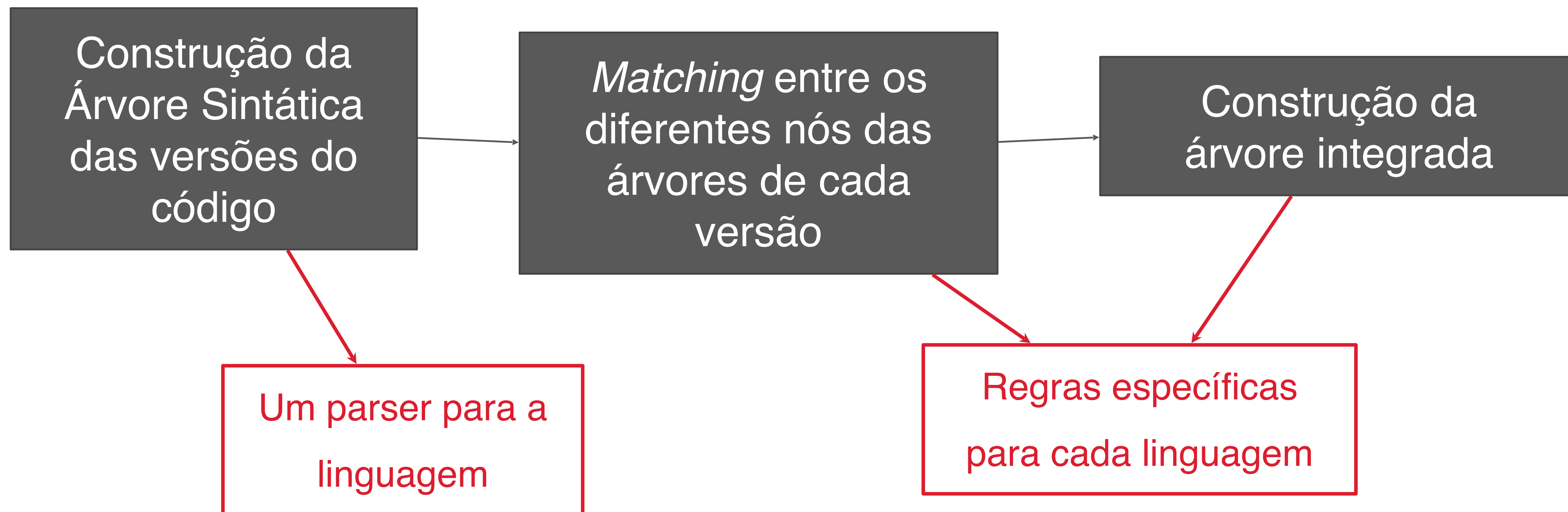
```
public class Account {  
    static void debit(int amount) {  
        // ...  
    }  
}
```

Merge

```
public class Account {  
    public static void debit(int  
amount) {  
        // ...  
    }  
}
```

Então, por que elas não são utilizadas na indústria?

Ferramentas existentes são específicas para uso com uma única linguagem.



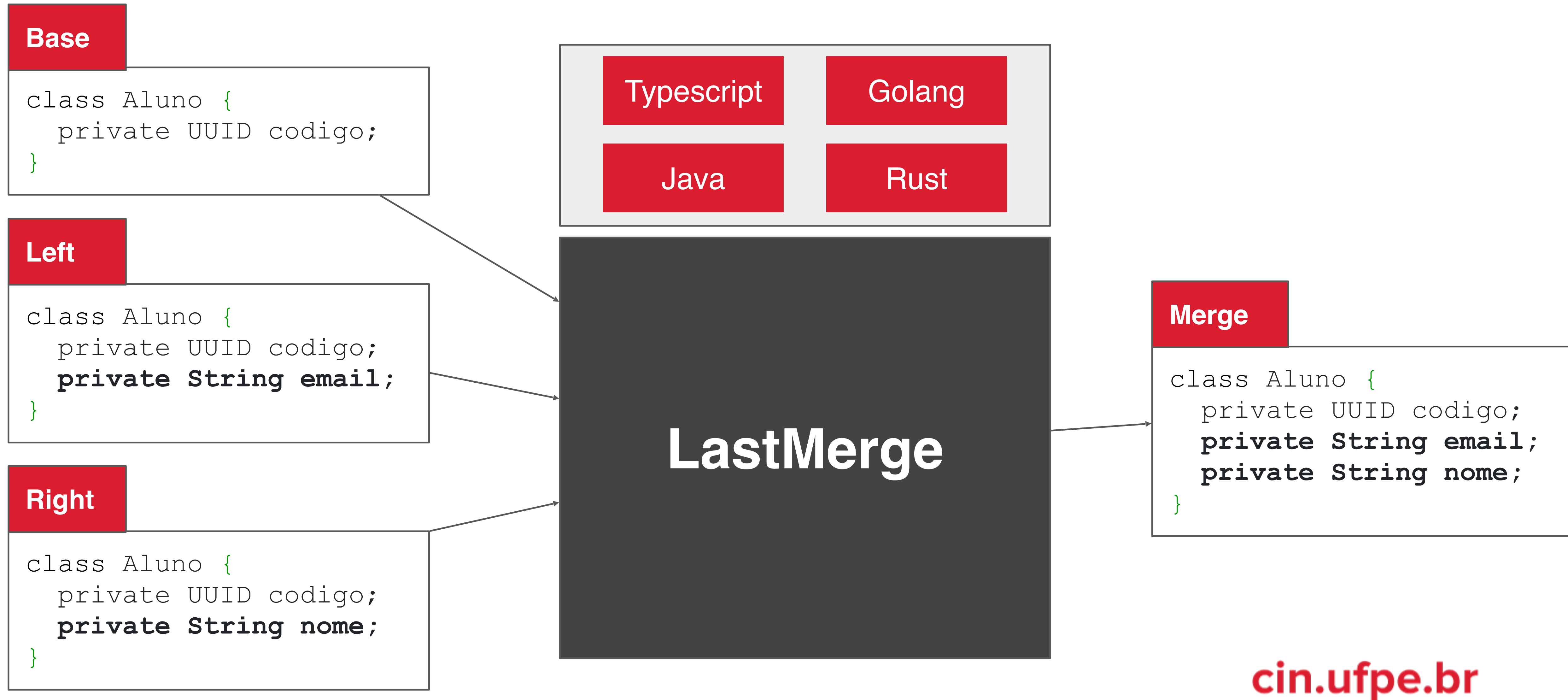
**Drawback 1: development and
maintenance costs of language
dependent tools**

**Drawback 2: computational
performance**

E se fosse possível isolar aspectos relacionados à linguagem?

Linguagem de
Programação

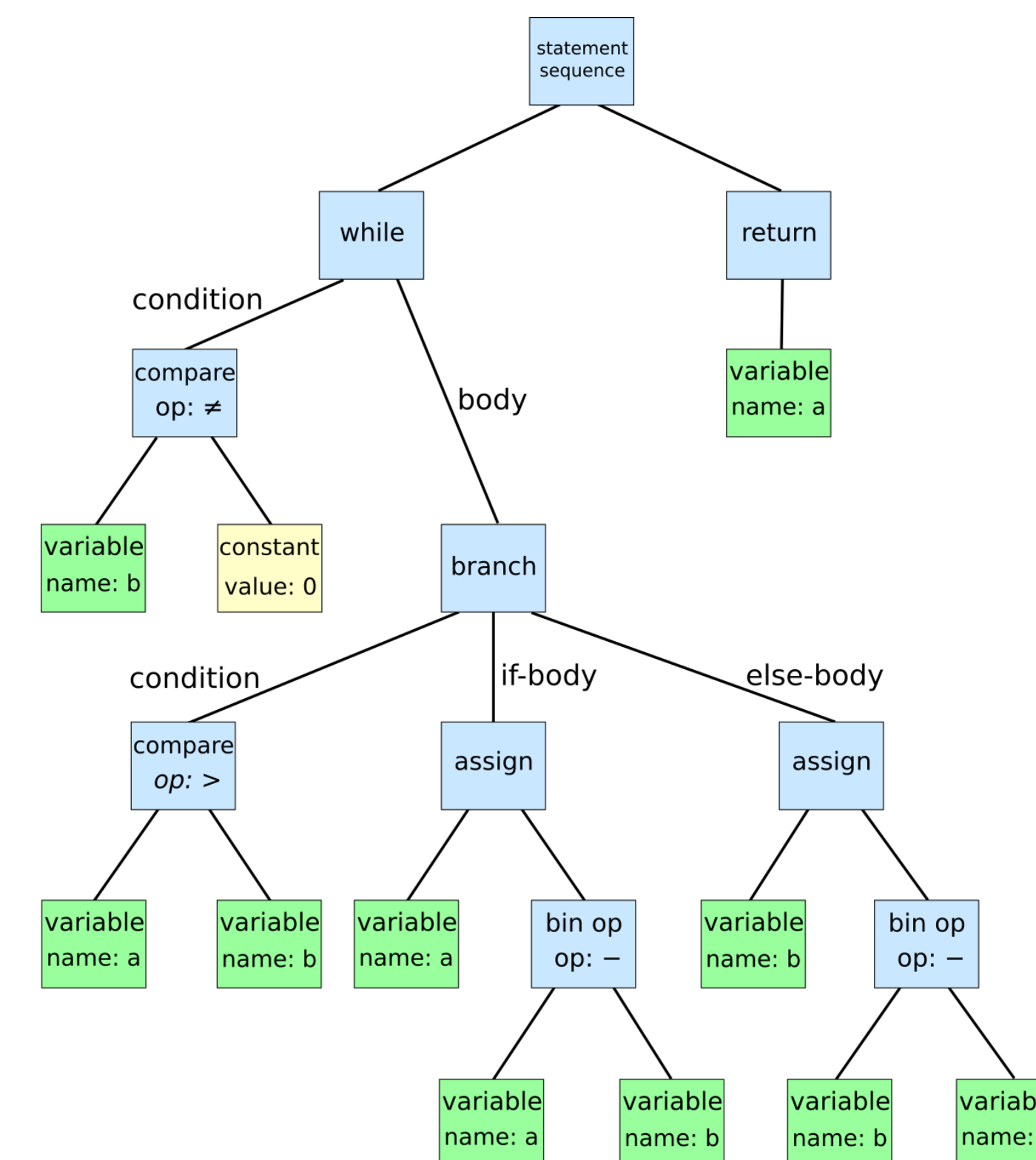
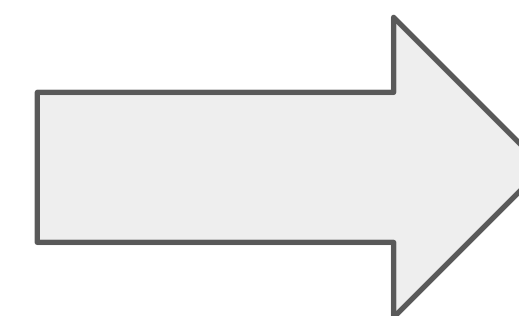
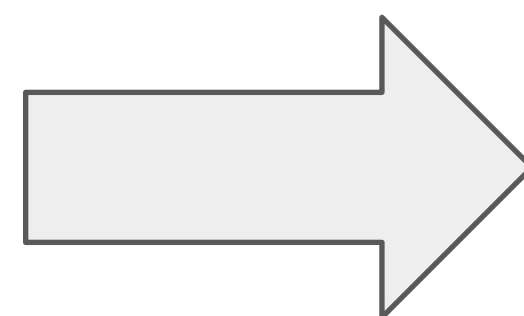
Language Agnostic Structured Tool for Code Merging



Generalizando o *parsing* com Tree Sitter

Código-Fonte

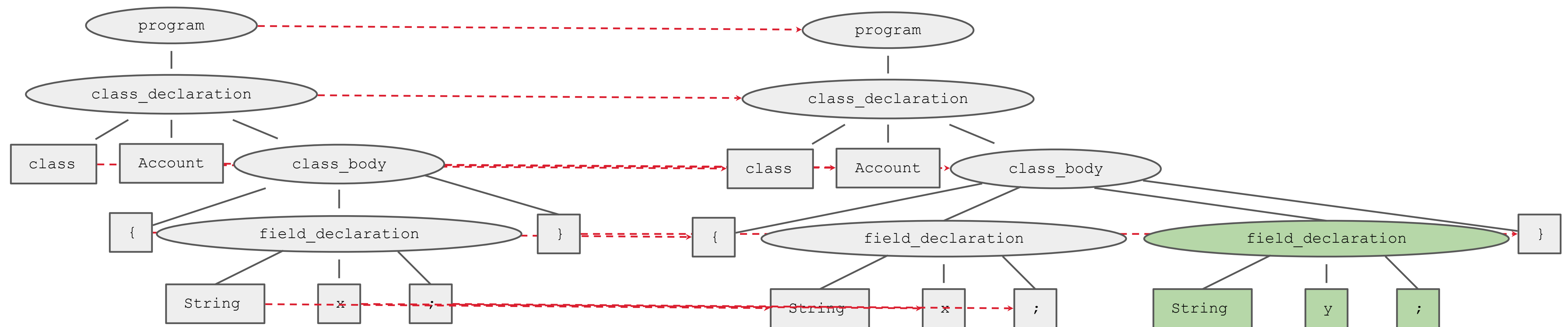
Descrição da
Gramática



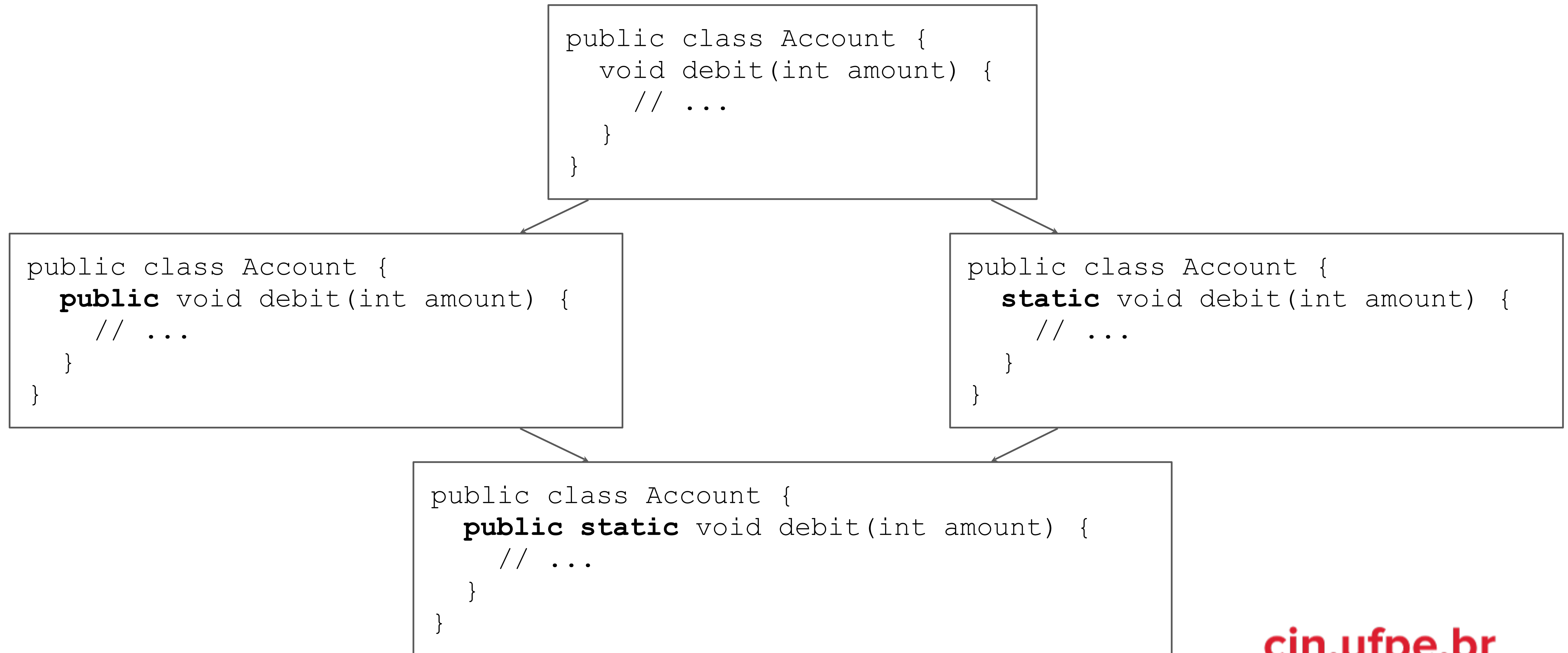
Instanciado para mais de 350 linguagens e usado em produção no GitHub

Matching: encontrando nós correspondentes entre as diferentes árvores

Mesmo algoritmos de jDime; Qual usar depende se podemos reordenar filhos; Level-wise



Merging: usando as informações de matching para inferir as operações realizadas



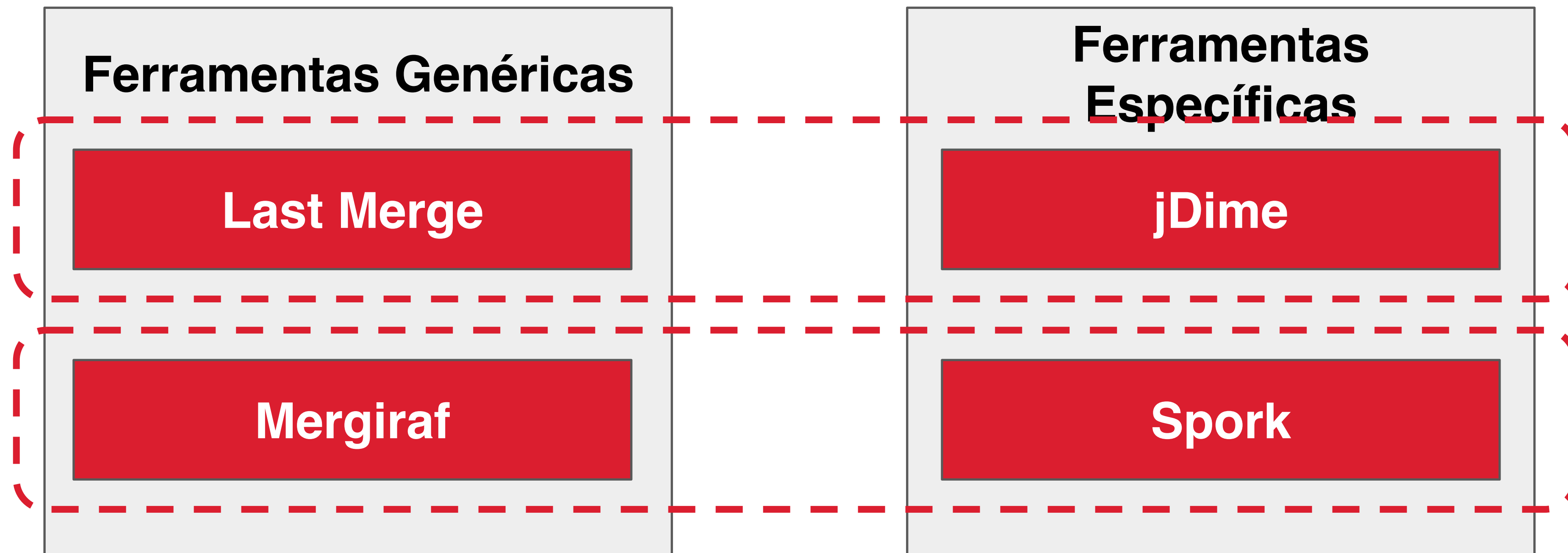
Mergiraf: outra ferramenta genérica

- Também utiliza o Tree Sitter para realizar o parsing do código.
- Reimplementa os algoritmos utilizados pela ferramenta específica Spork
 - GumTree para cálculo dos matchings entre as revisões
 - 3DM-Merge para merge das árvores.

Ferramentas genéricas
podem substituir
ferramentas específicas?

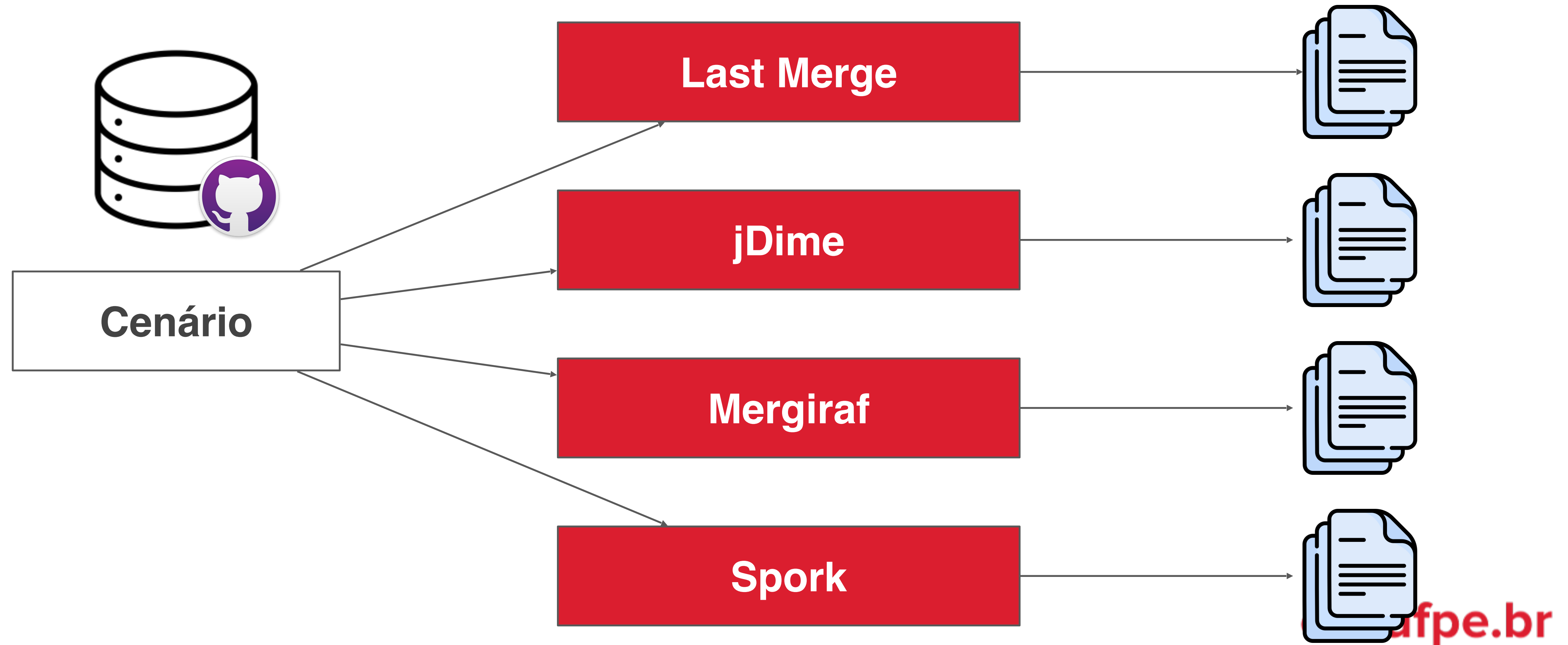
Estudo comparativo a fim de investigar

1. Como *merge* estruturado genérico impacta a acurácia?
2. *Merge* estruturado genérico é proibitivo computacionalmente?



Execução de cada ferramenta em cenários oriundos de Schesch et al. (2024)

5.229 cenários, abrangendo 13.675 arquivos; suítes de teste não triviais.

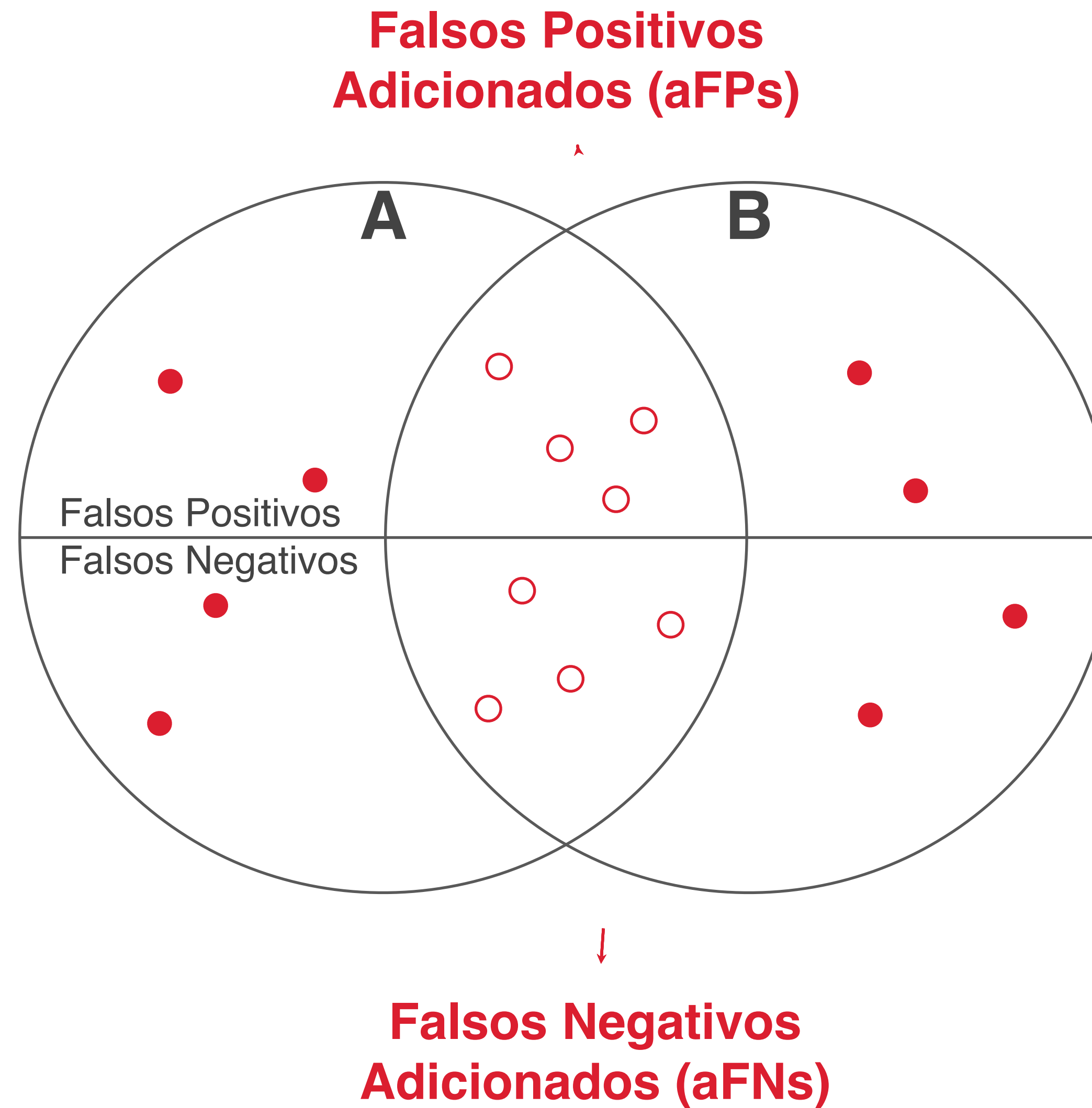


Como saber se a ferramenta integrou o cenário corretamente?

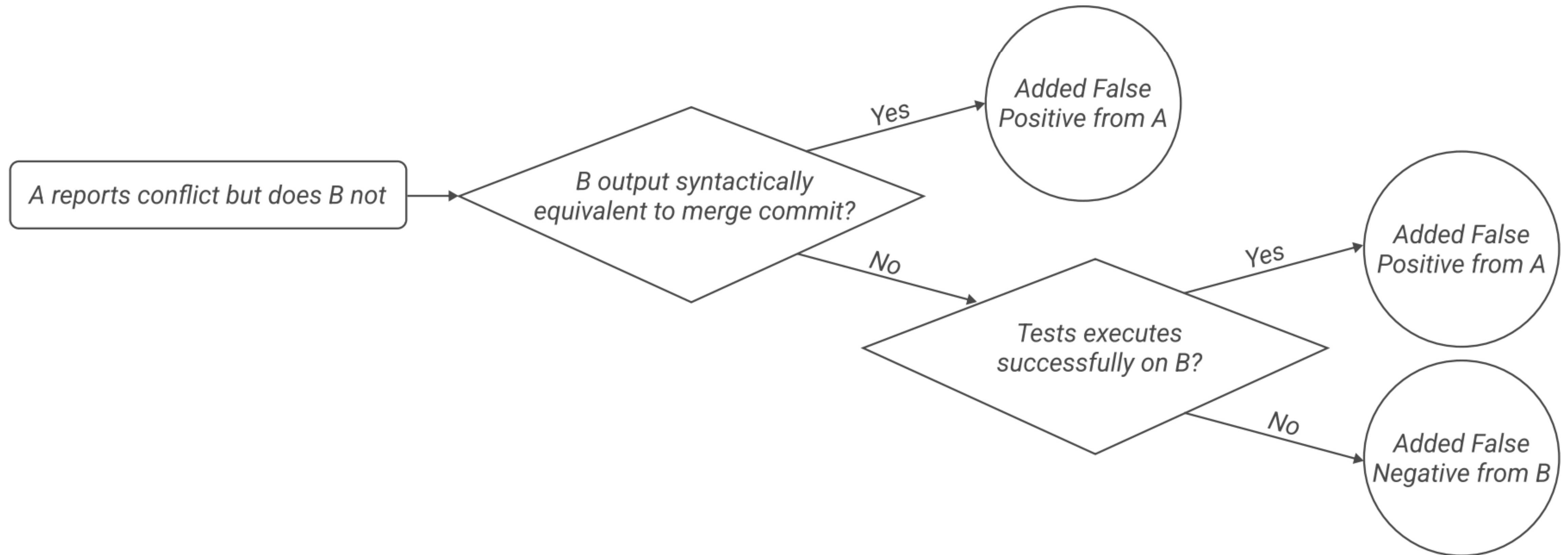
- Um **falso positivo** ocorre quando um conflito é reportado mesmo que, as modificações introduzidas não interfiram entre si;
- Um **falso negativo** ocorre quando as modificações introduzidas interferem entre si e nenhum conflito é reportado pela ferramenta.

Problema: detectar falsos positivos e negativos de maneira absoluta é difícil.

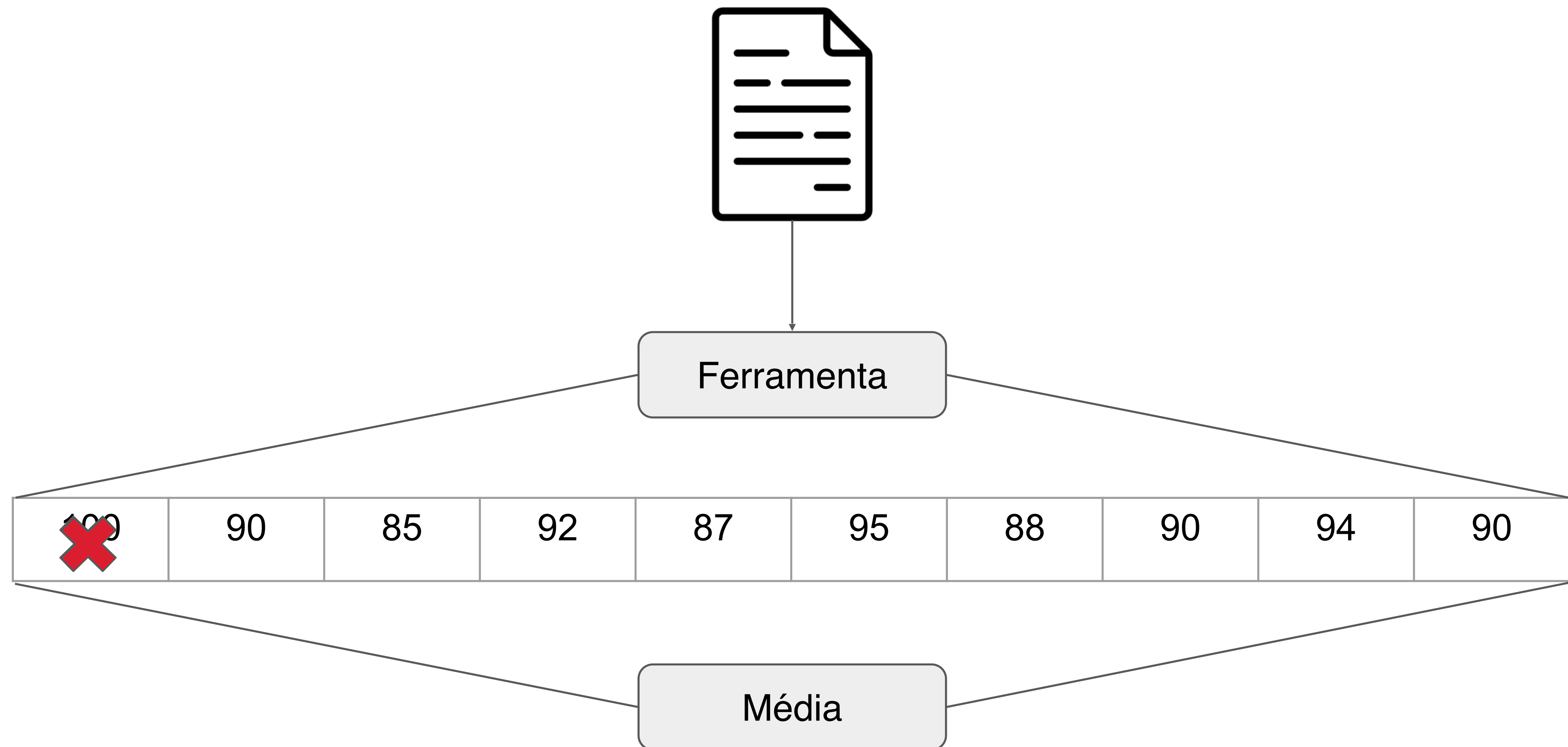
Comparação relativa: onde as ferramentas diferem



Utilizando testes e comparação sintática para identificar aFPs e aFNs



Respondendo RQ2: Computando o tempo de execução de um cenário



Em aproximadamente 10% dos cenários, genérico e específico discordam sobre a existência de conflitos.

Existência de conflitos	jDime vs LastMerge	Spork vs Mergiraf
Concordam	4909 (92,5%)	4697 (88,7%)
Discordam	400 (7,5%)	601 (11,3%)

Como essas diferenças impactam na acurácia do merge?

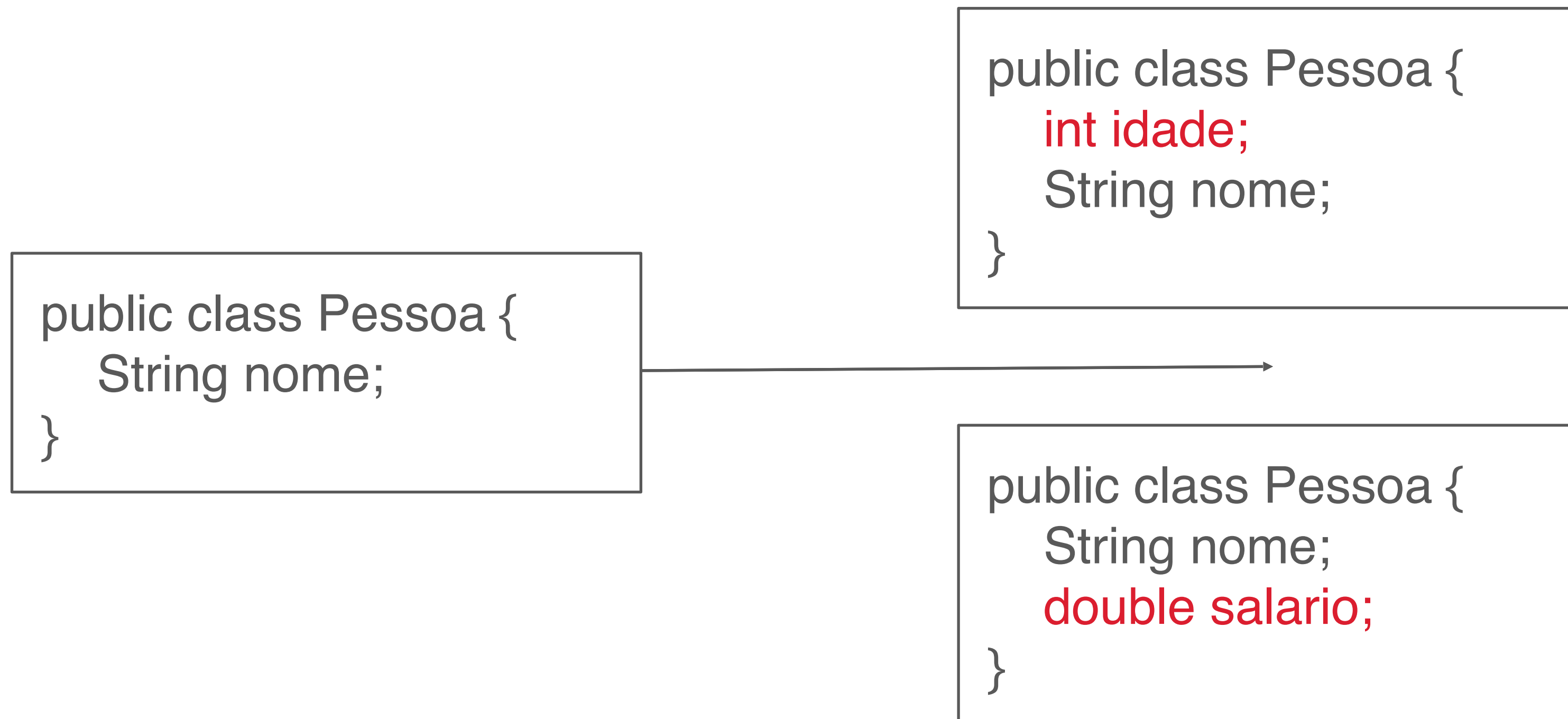
Por que essas diferenças aparecem?

Last Merge possui menos aFPs; jDime, menos aFNs

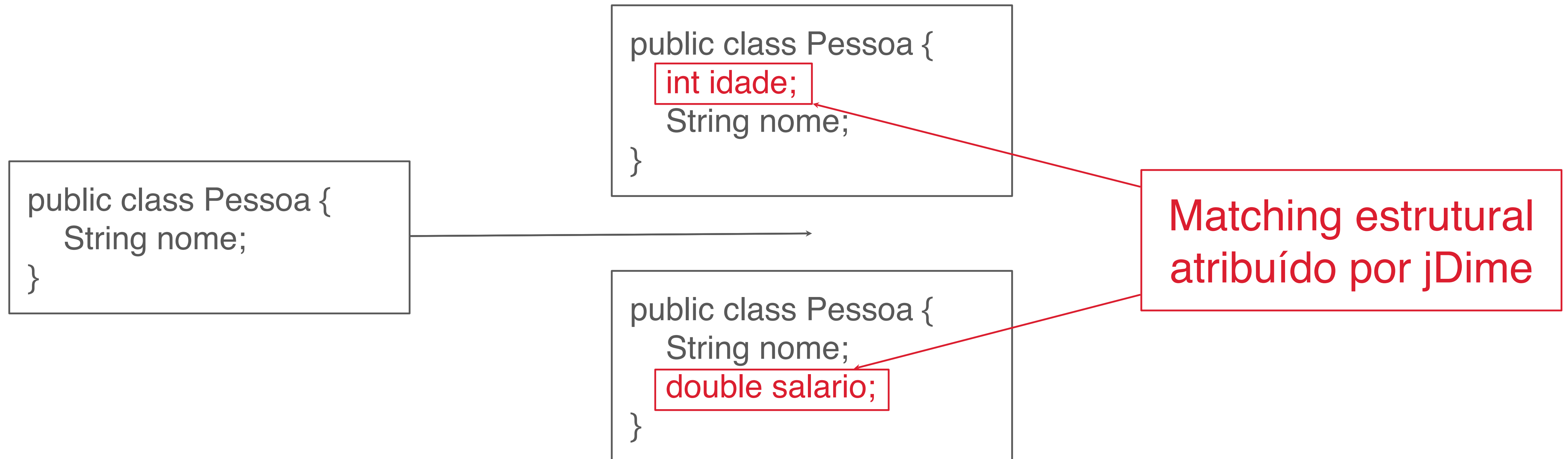
Situação	jDime	LastMerge
Falsos positivos adicionados (aFPs)	153	130
Falsos negativos adicionados (aFNs)	29	85

aFPs de jDime ocorrem por inacurácias na configuração

jDime só atribui identificadores à *imports* e *literais*; outros nós usam matching estrutural

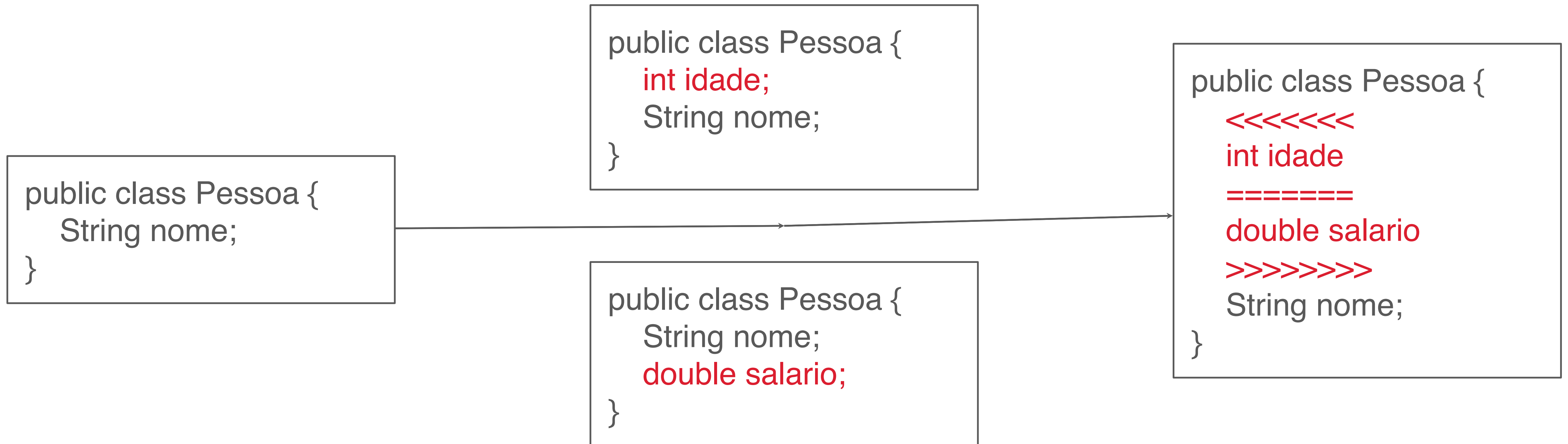


Por não atribuir identificadores, permite fazer o matching entre duas propriedades diferentes



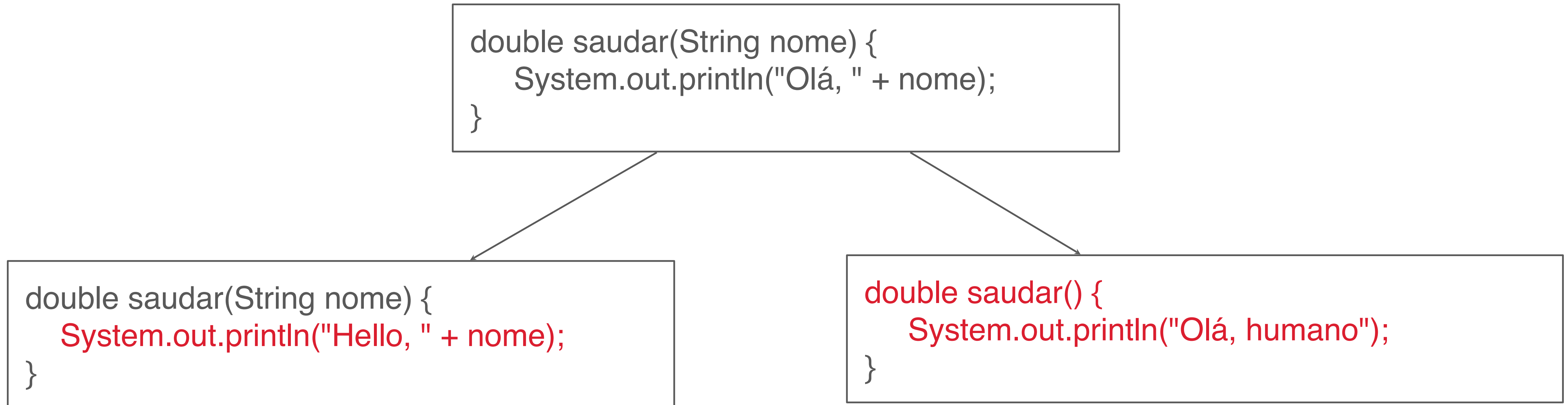
aFPs de jDime ocorrem por inacurácias na configuração

jDime só atribui identificadores a *imports* e *literais*



aFPs de Last Merge são de *renaming* (Cavalcanti et al, 2017)

Uma revisão modifica um método enquanto a outra modifica sua assinatura



Durante o matching, as modificações são incorretamente inferidas

```
double saudar(String nome) {  
    System.out.println("Olá, " + nome);  
}
```

```
double saudar(String nome) {  
    System.out.println("Hello, " + nome);  
}
```

```
double saudar() {  
    System.out.println("Olá, humano");  
}
```

Last Merge Matching

Edição do método saudar(String nome)

**Remoção do método saudar(String nome)
Adição do método saudar()**

Editar um nó que foi removido é um conflito

```
double saudar(String nome) {  
    System.out.println("Hello, " + nome);  
}
```

```
double saudar() {  
    System.out.println("Olá, humano");  
}
```

```
double saudar() {  
    System.out.println("Olá, humano");  
}
```

<<<<<<<<

```
double saudar(String nome) {  
    System.out.println("Olá, " + nome);  
}
```

=====

>>>>>>>

jDime evita aFPs de renaming, ao custo de aFNs

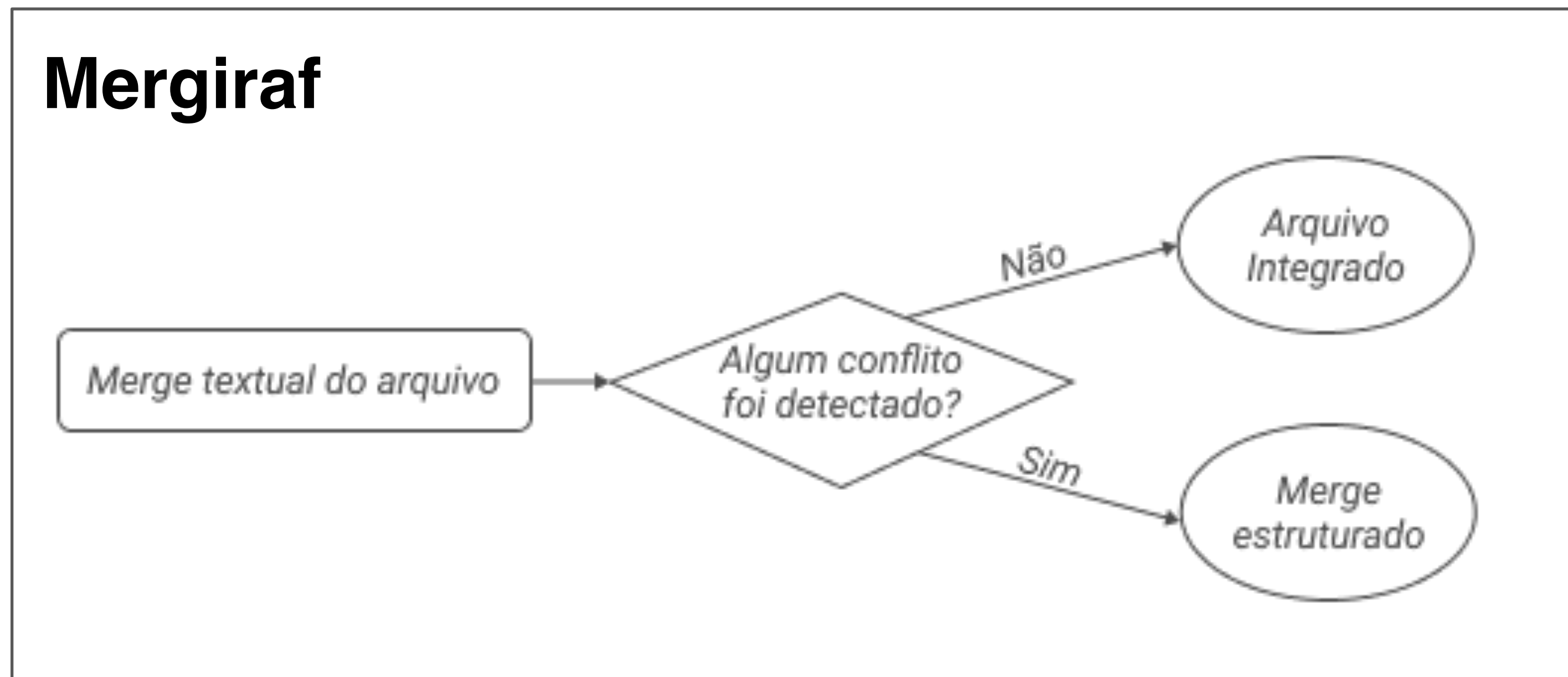


Last Merge detecta a interferência ao considerar toda a assinatura do construtor.

Olhando para Mergiraf e Spork, o cenário se inverte: Mergiraf possui mais aFPs; Spork, mais aFNs

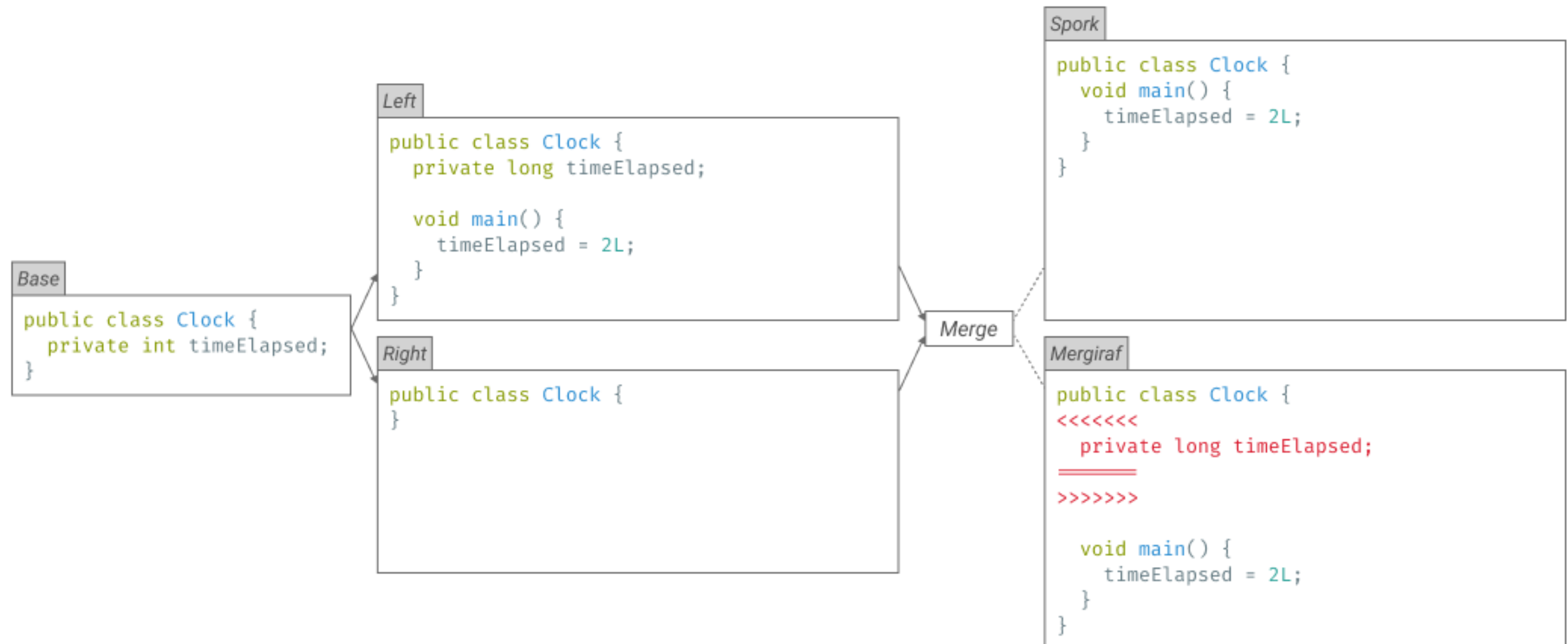
Situação	Spork	Mergiraf
Falsos positivos adicionados (aFPs)	150	290
Falsos negativos adicionados (aFNs)	107	62

A decisão de utilizar autotuning em Mergiraf explica a diferença nos aFPs de Spork

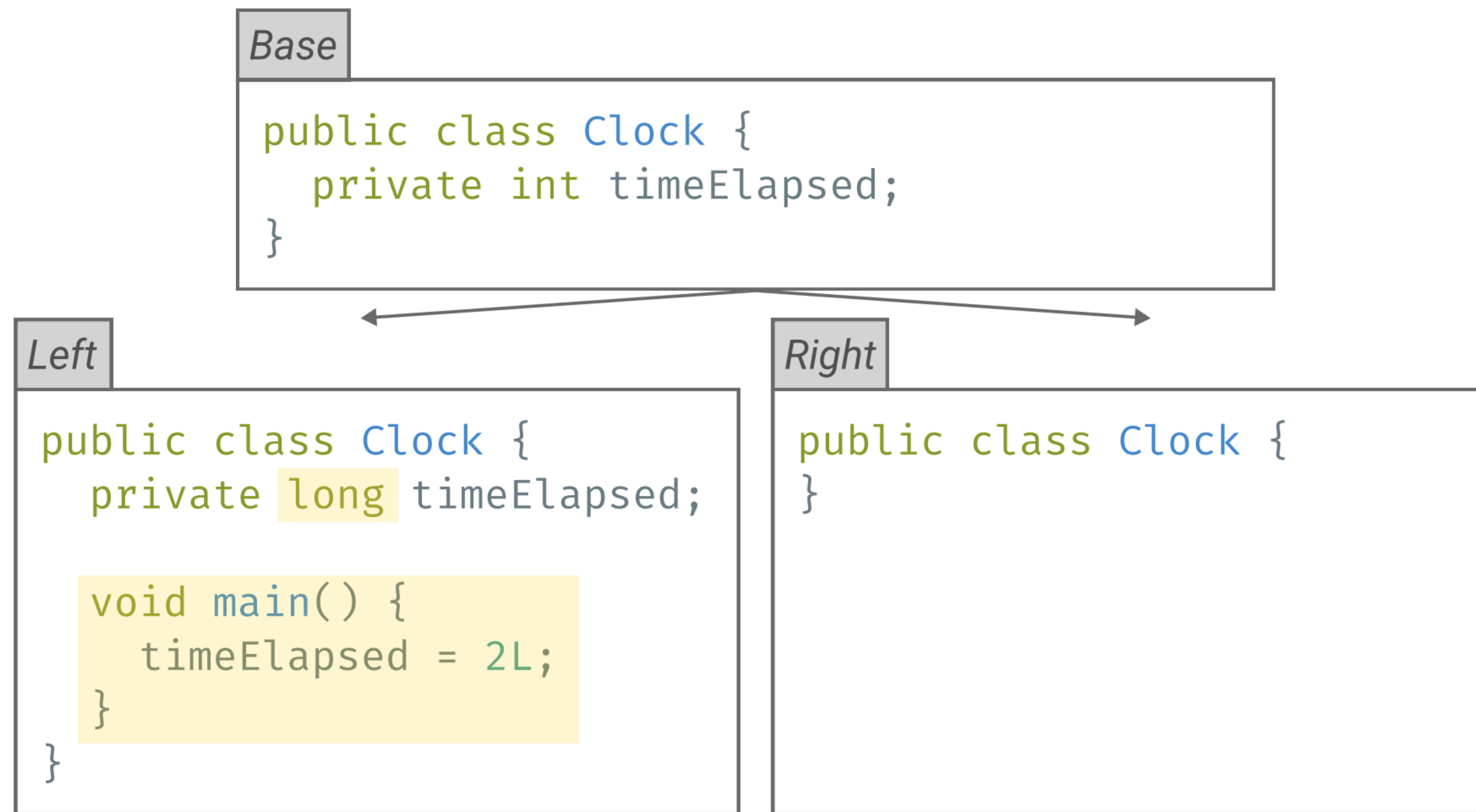


Em 4 dos 5 cenários analisados, Spork não conseguiu reproduzir, de maneira estruturada, o merge textual realizado por Mergiraf, resultando em conflitos.

Diferenças ocorrem por questões algorítmicas;
Spork não detecta conflitos de remoção/edição.



Diferenças ocorrem também por questões algorítmicas



Apesar de Right remover um nó editado por Left, Spork não reporta nenhum conflito

Spork

```
public class Clock {  
    void main() {  
        timeElapsed = 2L;  
    }  
}
```

Limitação da implementação de Spork do 3DM-MERGE, o algoritmo utilizado para integrar as modificações.

Mergiraf aperfeiçoa a implementação de 3DMMerge e é capaz de detectar o conflito

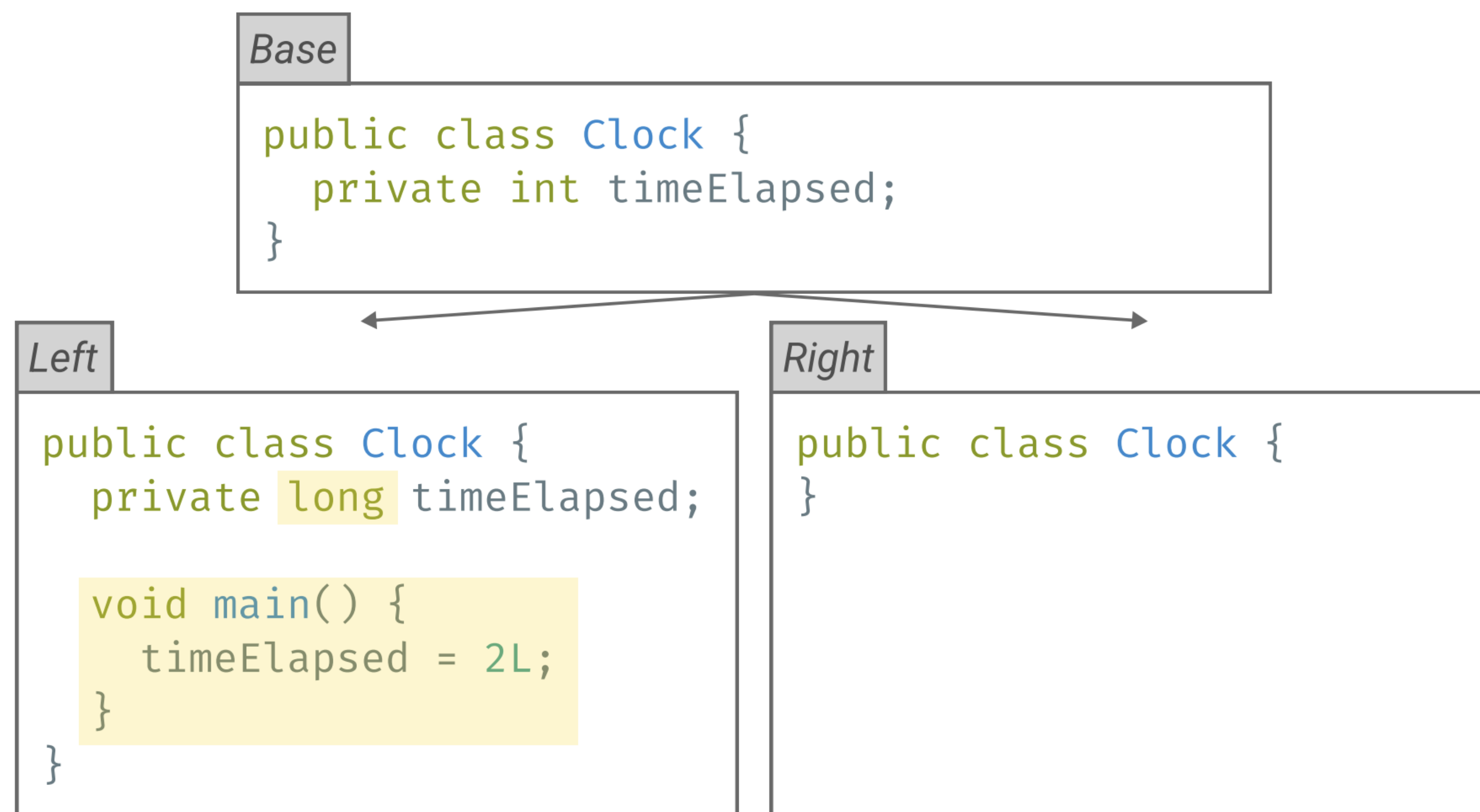
Mergiraf

```
public class Clock {  
<<<<<<<  
    private long timeElapsed;  
    =====  
>>>>>>>  
  
    void main() {  
        timeElapsed = 2L;  
    }  
}
```

Mergiraf rastreia quais nós foram removidos e editados em cada revisão.

Na prática, na maior parte das vezes, esse tipo de conflito é um falso positivo

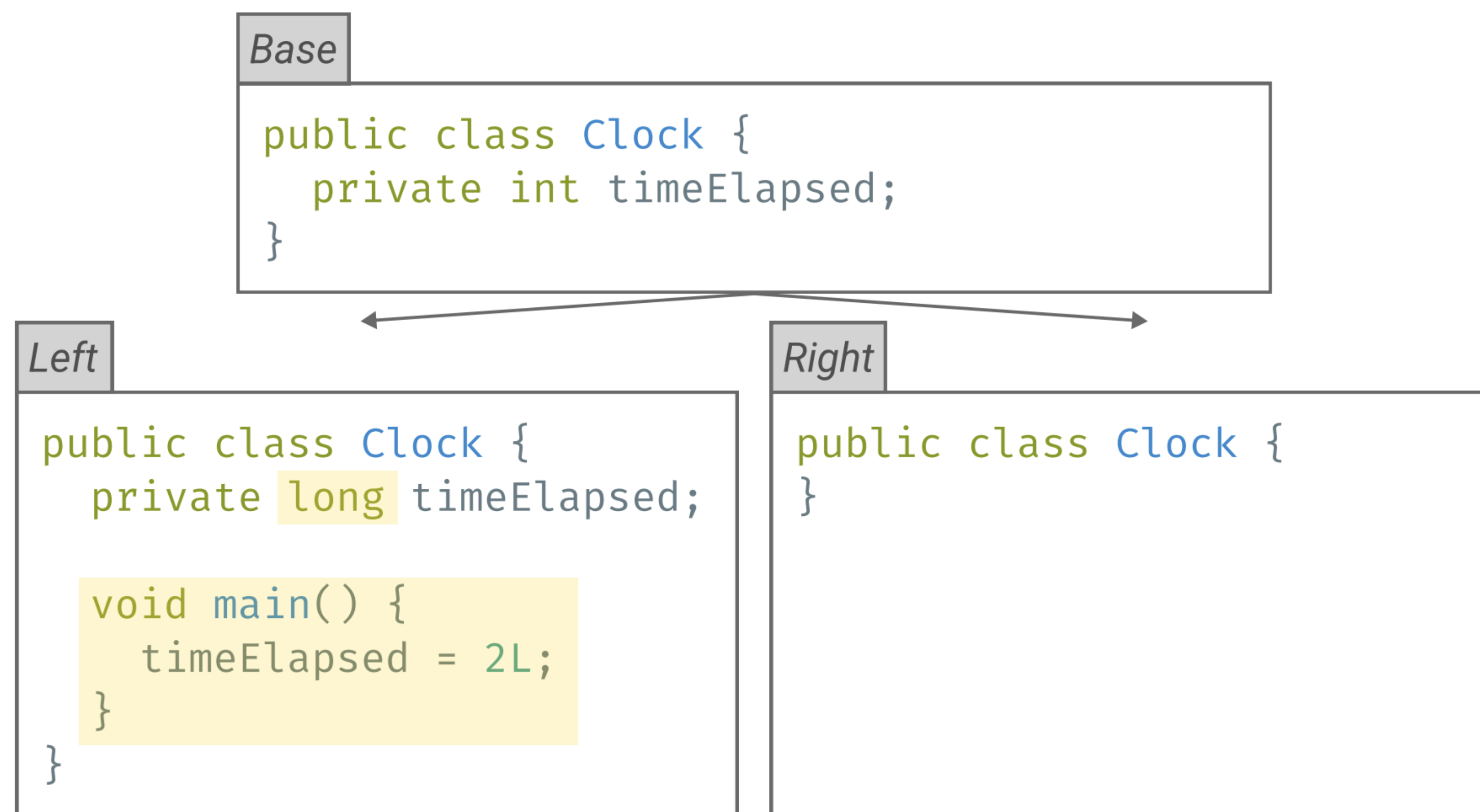
Somente se ainda existem referências ao nó removido após o merge, o build falha (conflito semântico).



Na prática, na maior parte das vezes, esse tipo de conflito é um falso positivo

Somente se ainda existem referências ao nó removido após o merge, o build falha (conflito semântico).

Explica a ocorrência de mais aFPs em Mergiraf que em Spork.



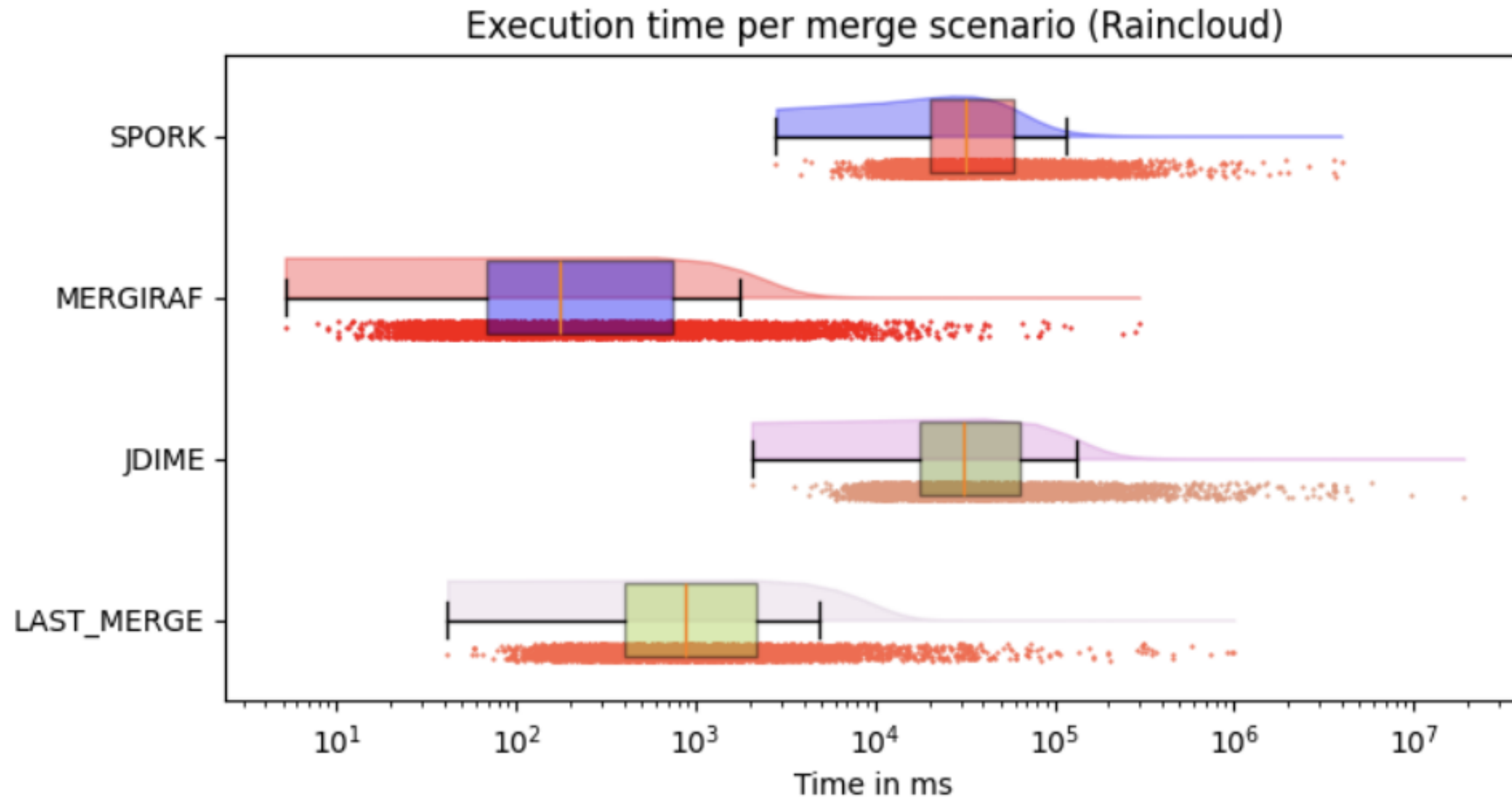
Nos dois casos, as diferenças observadas surgem por diferentes configurações e implementações

- Se as ferramentas tivessem a mesma configuração o resultado observado seria o mesmo;
- As diferenças existentes por implementação não são proibitivas: introduzir auto-tuning em Spork, por exemplo, seria trivial.

Mais importante: as
diferenças observadas **não**
estão relacionadas ao
requisito de
genericidade.

Sugere que ferramentas genéricas possuem acurácia semelhante à observada em ferramentas específicas

Além disso, introduzir genericidade não traz consigo um custo computacional proibitivo



Desempenho comparável, apesar das diferenças em design e implementação

- LastMerge e Mergiraf implementadas em Rust; Spork e jDime em Java.
- Mergiraf usa auto-tuning; melhora consideravelmente o desempenho.
- Mesmo considerando uma penalidade de 10x, ferramentas genéricas apresentam desenho comparável ao das específicas
- Uso de memória também não é proibitivo;
 - Cenário com 5500 LoCs, Peak Memory de Last Merge 35MB vs 825MB de jDime

Ferramentas genéricas
podem substituir as
específicas, sem prejuízo a
acurácia e a performance.

Certo, mas e para outras linguagens de programação?

- Nosso estudo focou em comparar ferramentas genéricas instanciadas para Java em comparação com ferramentas específicas;
- **Generalizar é difícil:** ferramentas estruturadas específicas para outras linguagens não estão disponíveis na literatura.

Quão extensível é LastMerge?

- Na prática, a ferramenta precisa ser configurável para uso com outras linguagens de programação;
- Além disso, a configuração deve ser fácil de fazer (plug and play);

Quão difícil é instanciar LastMerge para uso com outras linguagens de programação?

Construindo a interface de configuração: isolar os aspectos relacionados à linguagem de programação

Linguagem de
Programação

Gramática da linguagem

Quais nós podem permutar seus filhos?

Quais nós possuem identificadores?

Como extrair identificadores?

Utilizando casamento de padrão com Tree Sitter Queries

```
class Account {  
    // ...  
}
```

Parses to
→

```
(program  
  (class_declaration  
    name: (identifier)  
    body: (class_body)))
```

```
(class_declaration (identifier) @class_name)
```

Permitem encontrar padrões na árvore de maneira agnóstica a linguagem.

Instanciando Last Merge para C#

- Utilização da gramática oficial mantida pelo time do Tree Sitter.
- Identificação de nós que podem permutar filhos (classes, interfaces, etc.)
- Especificar como extrair identificadores para nós (assinaturas de métodos, nomes de classes e propriedades, etc.)

Tree Sitter Playground ajuda a facilitar o entendimento da árvore

```

1 class Account {
2     double currency;
3
4     bool withdrawn(double amount) {
5         // ...
6     }
7 }

```

Tree

0.2 ms

```

compilation_unit [0, 0] - [7, 0]
  class_declaration [0, 0] - [6, 1]
    name: identifier [0, 6] - [0, 13]
    body: declaration_list [0, 14] - [6, 1]
      field_declaration [1, 1] - [1, 17]
        variable_declaration [1, 1] - [1, 16]
          type: predefined_type [1, 1] - [1, 7]
          variable_declarator [1, 8] - [1, 16]
            name: identifier [1, 8] - [1, 16]
      method_declaration [3, 4] - [5, 5]
        returns: predefined_type [3, 4] - [3, 8]
        name: identifier [3, 9] - [3, 18]
        parameters: parameter_list [3, 18] - [3, 33]
          parameter [3, 19] - [3, 32]
            type: predefined_type [3, 19] - [3, 25]
            name: identifier [3, 26] - [3, 32]
        body: block [3, 34] - [5, 5]
          comment [4, 5] - [4, 11]

```

```

1 class Account {
2     private int amount = 0;
3 }

```

Query

```

1 (variable_declarator name: _ @name)

```

Especificação de queries

Identificação de nós

Percepção: instanciar Last Merge é simples

- Um desenvolvedor que conhece a linguagem deve levar de 2-4 horas;
- Configuração sucinta: PR que adiciona possui apenas 100 linhas;
- Mais importante: configuração pode ser feita de forma incremental
- Existem linguagens em que a instanciação é limitada?

Structured merge

Merge and Code Review

Paulo Borba
Informatics Center
Federal University of Pernambuco

pauloborba.cin.ufpe.br